

École Nationale des Sciences Appliquées de Tétouan

Filière BDIA1 — Big Data & Intelligence Artificielle



Système Intelligent de Contrôle Qualité des Impellers

*Détection automatique des défauts de fonderie
par Apprentissage Profond et Transfer Learning*

Rapport de projet de fin de module
Module : Apprentissage Profond et Vision par Ordinateur

Réalisé par :

Ayoub EL-MOUJAHID

Mohamed ERRAHMOUNI

Soulama Doubie Judicael Noe

Hazem DOUZI

Encadré par :

Prof. Dr Naoual ATTAOUI

Année universitaire **2025–2026**

Remerciements

Au terme de ce projet de fin de module, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui, par leur soutien, leurs conseils et leur disponibilité, ont contribué de près ou de loin à son aboutissement.

Nos remerciements les plus sincères s'adressent en premier lieu à notre encadrante, **Madame le Professeur Dr Naoual Attaoui**, pour la qualité de son encadrement, la rigueur scientifique de ses orientations et la confiance qu'elle nous a accordée tout au long de ce travail. Sa pédagogie et ses remarques éclairées ont constitué un guide précieux durant chaque étape de la réalisation.

Nous remercions également l'ensemble du corps professoral de la filière **BDIA1 (Big Data & Intelligence Artificielle)** pour la qualité des enseignements dispensés, qui ont jeté les fondations théoriques et pratiques sur lesquelles repose ce projet.

Notre reconnaissance va aussi à la communauté open-source — en particulier aux équipes de **TENSORFLOW**, **KERAS**, **PYTORCH**, **SCIKIT-LEARN** et **MATPLOTLIB** — ainsi qu'à Ravirajsinh Dabhi qui a publié sur Kaggle le jeu de données *Casting Defect Détection* sans lequel ce travail n'aurait pas été possible.

Enfin, nous adressons un remerciement chaleureux à nos familles et à nos camarades de promotion pour leur soutien moral indéfectible et l'enrichissement intellectuel mutuel qui a caractérisé notre année universitaire.

Les auteurs

Résumé

Dans le secteur de la fonderie, le contrôle qualité des pièces métalliques — en particulier des *impellers* (roues de pompes centrifuges) — représente un enjeu industriel majeur. Réalisé traditionnellement de manière visuelle par des opérateurs humains, ce contrôle souffre d’une triple limite : il est lent, subjectif et sujet à la fatigue. Une seule pièce défectueuse non détectée peut provoquer une défaillance en service avec des conséquences coûteuses pour le client final.

Le présent rapport propose un **système intelligent de contrôle qualité automatique** fondé sur la *vision par ordinateur* et l’*apprentissage profond*. Trois familles de modèles sont développées, comparées et discutées suivant une démarche progressive : (i) un **Perceptron Multi-Couches (MLP)** servant de *baseline*, (ii) un **Réseau de Neurones Convolutif (CNN)** entraîné de zero, et (iii) un modèle de **Transfer Learning** fondé sur l’architecture EFFICIENTNETB0 pre-entraînée sur IMAGENET.

Une attention particulière est portée à la chaîne de prétraitement, développée en double implémentation TENSORFLOW (`tf.data`) et PYTORCH (`DataLoader`) afin de garantir reproductibilité, modularité et portabilité. Cette chaîne intègre la normalisation *ImageNet*, des transformations *color balancing* (white balance, equalisation d’histogramme, channel shuffle), des augmentations géométriques et photométriques, ainsi qu’une gestion explicite du déséquilibre des classes par *class weights* ou `WeightedRandomSampler`.

Le modèle final atteint une **accuracy de 99,44%** et un **AUC de 0,9993** sur l’ensemble de test, avec un *recall* de **99,34%** sur la classe défectueuse — métrique critique en contrôle qualité industriel. Ces performances, obtenues avec seulement 6 600 images d’entraînement, démontrent la pertinence du *Transfer Learning* pour les problèmes de classification industrielle à faible volume de données.

Mots-clés : Contrôle qualité, Fonderie, Impeller, Apprentissage profond, Réseaux de neurones convolutifs, Transfer Learning, EfficientNet, TensorFlow, PyTorch, Vision par ordinateur, Détection de défauts.

Abstract

In the foundry industry, quality control of metallic parts — particularly *impellers* (centrifugal pump wheels) — represents a major industrial challenge. Traditionally performed visually by human operators, such inspection suffers from three intrinsic limitations : it is slow, subjective, and prone to fatigue-induced errors. A single undetected defective part can lead to in-service failure with costly conséquences for the end customer.

This report proposes an **intelligent automatic quality control system** based on *computer vision* and *deep learning*. Three model families are developed, compared, and discussed following a progressive approach : (i) a **Multi-Layer Perceptron (MLP)** as a baseline, (ii) a **Convolutional Neural Network (CNN)** trained from scratch, and (iii) a **Transfer Learning** model based on the EFFICIENTNETB0 architecture pre-trained on IMAGENET.

Special attention is devoted to the preprocessing pipeline, developed in dual implémentation TENSORFLOW (`tf.data`) and PYTORCH (`DataLoader`) to ensure reproducibility, modularity and portability. The pipeline integrates *ImageNet* normalization, *color balancing* transformations (white balance, histogram equalization, channel shuffle), geometric and photometric augmentations, as well as explicit class imbalance handling through *class weights* or `WeightedRandomSampler`.

The final model achieves **99.44% accuracy** and an **AUC of 0.9993** on the test set, with **99.34% recall** on the defective class — the critical metric in industrial quality control. These results, obtained with only 6,600 training images, demonstrate the relevance of *Transfer Learning* for industrial classification problems with limited data.

Keywords : Quality Control, Foundry, Impeller, Deep Learning, Convolutional Neural Networks, Transfer Learning, EfficientNet, TensorFlow, PyTorch, Computer Vision, Defect Détection.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Liste des figures	8
Liste des tableaux	9
Liste des acronymes	10
1 Introduction générale	11
1.1 Contexte général	11
1.2 Problematique	11
1.3 Objectifs du projet	12
1.4 Méthodologie générale	12
1.5 Organisation du rapport	13
2 État de l'art	14
2.1 Contrôle qualité industriel par vision artificielle	14
2.2 Architectures convolutives marquantes	14
2.2.1 Les jalons historiques	14
2.2.2 EfficientNet : état de l'art récent	15
2.3 Transfer Learning	15
2.4 Travaux relatifs à la détection de défauts en fonderie	16
3 Procédé de fabrication et typologie des défauts	17
3.1 Procédé de fonderie en moule carapace	17
3.1.1 Étape 1 : Conception (Design of pump)	17
3.1.2 Étape 2 : Fabrication du moule carapace (Shell mould)	17
3.1.3 Étape 3 : Coulée (Shell mould casting process)	17
3.2 Typologie des défauts observés	18
3.3 Objectif spécifique du projet	19
4 Acquisition et organisation des données	20
4.1 Choix du jeu de données	20

4.2	Description du dataset original	20
4.3	Répartition train / validation / test	21
4.4	Volumes et déséquilibre des classes	22
4.5	Vers la version augmentée pour le Transfer Learning	23
5	Prétraitement des images	24
5.1	Philosophie générale du pipeline	24
5.2	Vue d'ensemble du pipeline TensorFlow (10 sections)	24
5.2.1	Reproductibilité et détection du <i>device</i>	25
5.2.2	Color balancing : compatibilité <i>graph</i> TensorFlow	26
5.2.3	Normalisation ImageNet	27
5.2.4	Comparaison train vs eval	27
5.2.5	Chargement et pondération des classes	28
5.2.6	Composition finale des datasets	29
5.2.7	Configuration centralisée et utilitaires	29
5.3	Pipeline équivalent en PyTorch	30
5.3.1	Reproductibilité et device	30
5.3.2	Color balancing personnalisée	31
5.3.3	Pipeline complet d'augmentations	31
5.3.4	Gestion du déséquilibre des classes	32
5.3.5	DataLoaders et utilitaires	32
6	Modélisation et expérimentations	34
6.1	Modèle 1 : Perceptron Multi-Couches (MLP)	34
6.1.1	Motivation	34
6.1.2	Représentation des données : le Flatten	34
6.1.3	Architecture du MLP	35
6.1.4	Mécanismes de régularisation	36
6.1.5	Résultats du MLP	36
6.1.6	Analyse critique du MLP	37
6.2	Modèle 2 : CNN entraîné de zéro	37
6.2.1	Principe et fondements théoriques	38
6.2.2	Architecture du CNN proposé	38
6.2.3	Compilation et régularisation	39
6.2.4	Entraînement	39
6.2.5	Évaluation et diagnostic d'overfitting	40
6.2.6	Régularisation par augmentation des données	41
6.2.7	Conclusion intermédiaire sur le CNN	42
6.3	Modèle 3 : Transfer Learning avec EfficientNetB0	42
6.3.1	Principe du Transfer Learning	42

6.3.2	Stratégie en deux phases	43
6.3.3	Architecture choisie : EfficientNetB0	44
6.3.4	Source des données et preparation	45
6.3.5	Hyperparamètres	47
6.3.6	Construction du modèle	47
6.3.7	Phase 1 : Extraction de caractéristiques	48
6.3.8	Phase 2 : Fine-tuning	48
6.3.9	Courbes d'entraînement consolidées	49
7	Évaluation finale et discussion	51
7.1	Protocole d'évaluation	51
7.2	Métriques industrielles clés	51
7.2.1	Rappel des definitions	51
7.2.2	Résultats sur le test set	52
7.2.3	Asymétrie des coûts d'erreur	52
7.3	Matrice de confusion	53
7.4	Courbes ROC et Precision-Recall	53
7.5	Analyse fine des erreurs	54
7.6	Comparaison globale des trois modèles	55
7.7	Discussion industrielle	56
7.7.1	Adequation au déploiement	56
7.7.2	Limites résiduelles	56
8	Conclusion générale et perspectives	58
8.1	Bilan du projet	58
8.2	Apports personnels	59
8.3	Perspectives	59
8.3.1	Extension fonctionnelle	59
8.3.2	Industrialisation	60
8.3.3	Amelioration continue	60
8.3.4	Ouverture vers l'Industrie 4.0	60
	Bibliographie	63

Table des figures

3.1	Étapes du procédé de fonderie en moule carapace.	18
3.2	Exemples typiques de défauts comparés à une pièce conforme.	19
4.1	Aperçu visuel du dataset <i>Casting Defect Detection</i> (Kaggle, version originale).	21
4.2	Distribution des classes par sous-ensemble (train / validation / test).	22
5.1	Bandeau de présentation du pipeline de prétraitement TensorFlow.	25
5.2	Sorties des utilitaires de contrôle du pipeline.	30
6.1	Aplatissement (<i>Flatten</i>) d'une image en vecteur 1D.	34
6.2	Schéma global de l'architecture MLP.	35
6.3	Performance du MLP sur le jeu de test.	36
6.4	Architecture du CNN entraîne de zero.	38
6.5	Compilation et configuration du CNN avec callbacks de régularisation.	39
6.6	Entraînement du CNN sur 30 epoques.	40
6.7	Courbes d'entraînement du CNN avant data augmentation : signe clair de sur-apprentissage.	40
6.8	Effet de la <i>data augmentation</i> sur l'apprentissage.	41
6.9	Synthèse visuelle des résultats du CNN.	42
6.10	Principe général du Transfer Learning : réutilisation des features d'un modèle pre-entraîne.	43
6.11	Réutilisation du <i>backbone</i> pre-entraîne : la tête originale (1000 classes ImageNet) est remplacée par une tête binaire.	44
6.12	Architecture détaillée d'EfficientNetB0 (blocs MBConv).	45
6.13	Dataset <i>Casting Defect Détection</i> dans sa version augmentée fournie sur Kaggle.	45
6.14	Code de chargement et de prétraitement adapté à EfficientNet.	46
6.15	Hyperparamètres clés utilisés pour les deux phases.	47
6.16	Construction du modèle de Transfer Learning.	47
6.17	Logs d'entraînement Phase 1 - Extraction de caractéristiques.	48
6.18	Logs d'entraînement Phase 2 - Fine-tuning.	49
6.19	Courbes <i>accuracy</i> , <i>loss</i> et AUC durant les phases 1 et 2.	49
7.1	Performance détaillée sur l'ensemble de test (715 images).	52
7.2	Matrice de confusion sur le <i>test set</i>	53
7.3	Courbes ROC (gauche) et Precision-Recall (droite) sur le test set.	53

7.4	Visualisation des images mal classées : FN à gauche, FP à droite.	55
7.5	Comparaison des courbes ROC — le Transfer Learning (rouge) discrimine quasi-parfaitement les deux classes, quel que soit le seuil.	56
8.1	Synthèse visuelle du résultat final.	58

Liste des tableaux

4.1	Répartition train / validation / test du dataset.	21
4.2	Volumes du dataset original <i>Casting Defect Detection</i> par split et par classe.	22
5.1	Constantes globales partagées par tous les notebooks.	26
5.2	Comparaison des transformations <i>train</i> vs <i>eval</i>	28
5.3	Configuration des trois DataLoaders PyTorch.	32
6.1	Résultats du MLP sur l'ensemble de test.	37
6.2	Stratégie de Transfer Learning à deux étages.	43
7.1	Comparatif des trois approches. [†] après <i>data augmentation</i>	55

Liste des acronymes

AUC Area Under the Curve.

CAO Conception Assistée par Ordinateur.

CNN Convolutional Neural Network.

FN Faux Negatif.

FP Faux Positif.

GPU Graphics Processing Unit.

MLP Multi-Layer Perceptron.

ReLU Rectified Linear Unit.

TFLite TensorFlow Lite.

TL Transfer Learning (apprentissage par transfert).

1. Introduction générale

1.1 Contexte général

La quatrième révolution industrielle, communément désignée sous le terme *Industrie 4.0*, redéfinit en profondeur les processus de production manufacturière. Au cœur de cette transformation se trouve l'intégration massive de technologies numériques — objets connectés, jumeaux numériques, robotique collaborative et, plus particulièrement, *intelligence artificielle* — au sein des chaînes de fabrication. Dans ce contexte, le **contrôle qualité automatique** apparaît comme l'une des applications à plus forte valeur ajoutée : il conditionne directement la fiabilité des produits livrés, la satisfaction client et la rentabilité des installations.

La fonderie, secteur historique et stratégique de l'industrie métallurgique, n'échappe pas à cette mutation. Les pièces issues du moulage — particulièrement les composants critiques tels que les *impellers* de pompes centrifuges — doivent satisfaire à des spécifications dimensionnelles et de surface très exigeantes. Tout écart (fissure, déformation, bavure, porosité) peut compromettre la fonction hydraulique de la pompe et entraîner une défaillance prématurée, parfois catastrophique, en service.

1.2 Problématique

Le contrôle qualité des impellers en sortie de chaîne de fonderie est aujourd'hui réalisé selon deux approches :

- **Inspection visuelle manuelle.** Un opérateur examine chaque pièce à l'œil nu et la classe comme conforme ou défectueuse. Cette méthode est universelle mais souffre de trois limites majeures : *lenteur* (cadence de production limitée), *subjectivité* (variabilité inter-opérateurs et intra-opérateur), et *fatigue* (taux d'erreur croissant en fin de poste).
- **Contrôle non destructif (CND) instrumenté.** Radiographie, ressuage, ultrasons. Très précis mais coûteux, lents et inadaptés à un contrôle de masse à 100 %.

Aucune de ces deux approches ne permet à la fois un contrôle *exhaustif*, *rapide* et *reproductible* en ligne. Cette limitation conduit, dans la pratique industrielle, à l'application

d'un contrôle par échantillonnage statistique — ce qui laisse mécaniquement passer une fraction de pièces défectueuses chez le client.

Question de recherche

Est-il possible de concevoir un système automatique, capable de classifier en quelques millisecondes une image de pièce comme **conforme** ou **défectueuse**, avec une précision suffisante pour un déploiement industriel et un *recall* élevé sur la classe défectueuse afin de minimiser les faux négatifs ?

1.3 Objectifs du projet

Pour répondre à cette problématique, le présent projet poursuit les objectifs suivants :

1. **Étudier** les principes et l'état de l'art de la classification d'images par apprentissage profond, en particulier les architectures convolutives et le *Transfer Learning*.
2. **Concevoir** une chaîne de prétraitement reproductible et portable, adaptée aux contraintes des images industrielles (variations d'éclairage, angles, textures de surface).
3. **Implementer et comparer** trois modèles de classification représentatifs d'une progression méthodologique : Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN) et Transfer Learning sur EFFICIENTNETB0.
4. **Evaluer rigoureusement** les performances obtenues à l'aide de métriques industriellement pertinentes (*accuracy*, *précision*, *recall*, F1-score, Area Under the Curve (AUC)) et d'une analyse fine des erreurs (faux négatifs, faux positifs).
5. **Discuter** les limites du système propose et formuler des perspectives pour son industrialisation effective (déploiement embarqué, multi-classes, localisation).

1.4 Méthodologie générale

La démarche méthodologique adoptée s'inscrit dans une logique *incrémentale* et *comparative* :

1. **Acquisition** d'un jeu de données public reconnu (*Casting Defect Détection*, Kaggle).
2. **Exploration et prétraitement** avec une double implémentation TensorFlow / PyTorch pour valider la robustesse du pipeline.

3. **Modélisation progressive** : chaque modèle est entraîné, évalué, ses limites identifiées, et la génération suivante est conçue pour les dépasser.
4. **Évaluation finale** sur un ensemble de test *strictement disjoint*, jamais vu durant l'apprentissage.
5. **Analyse critique** des résultats et des erreurs résiduelles dans une perspective de déploiement industriel.

1.5 Organisation du rapport

Ce rapport est structuré en **huit chapitres** :

Chapitre 1

La présente introduction générale.

Chapitre 2

État de l'art du contrôle qualité par vision artificielle et apprentissage profond.

Chapitre 3

Procédé de fonderie en moule carapace et typologie des défauts observés sur les impellers.

Chapitre 4

Acquisition, organisation et exploration du jeu de données.

Chapitre 5

Conception détaillée du pipeline de prétraitement (TensorFlow et PyTorch).

Chapitre 6

Modélisation : MLP, CNN et Transfer Learning avec EFFICIENTNETB0.

Chapitre 7

Évaluation finale, analyse des erreurs et discussion industrielle.

Chapitre 8

Conclusion générale et perspectives.

2. État de l’art

2.1 Contrôle qualité industriel par vision artificielle

Le contrôle qualité par vision artificielle (*Machine Vision*) est une discipline ancienne, dont les premiers systèmes industriels remontent aux années 1980. Historiquement, ces systèmes reposaient sur des *règles expertes* fondées sur le traitement d’image classique : seuillage adaptatif, détection de contours (filtres de Sobel, Canny), morphologie mathématique, et descripteurs de texture (matrices de co-occurrence, ondelettes de Gabor). Ces approches, performantes sur des défauts simples et bien définis, montrent rapidement leurs limites dès que l’on s’attaque à des défauts *visuellement variables* — tels que les fissures fines, les déformations subtiles ou les bavures hétérogènes que l’on rencontre en fonderie.

L’avenement de l’apprentissage profond, et plus précisément des réseaux convolutifs [1], a profondément bouleverse ce paysage. La capacité des CNN à apprendre automatiquement des représentations hiérarchiques des images — des contours de bas niveau aux concepts sémantiques de haut niveau — a permis des progrès considérables, comme illustre par la victoire d’ALEXNET [2] à la compétition ILSVRC 2012 sur IMAGENET. Depuis, le *deep learning* s’est imposé comme l’approche dominante pour la classification d’images, et son adoption dans le contrôle qualité industriel s’est rapidement généralisée.

2.2 Architectures convolutives marquantes

2.2.1 Les jalons historiques

- **LeNet-5** [1] : premier CNN moderne, appliqué à la reconnaissance de chiffres manuscrits. Pose les bases (convolution, pooling, couches denses).
- **AlexNet** [2] : 8 couches, exploitation du GPU, fonctions d’activation Rectified Linear Unit (ReLU), *dropout*. Marque le début de l’ère *deep*.
- **VGGNet** [3] : empilement profond (16 à 19 couches) de petits filtres 3×3 . Démonstration que la profondeur améliore la performance.
- **ResNet** [5] : introduction des connexions résiduelles (*skip connections*), permettant d’entraîner des réseaux jusqu’à 152 couches sans dégradation.
- **Inception / GoogLeNet** [4] : *Inception modules* parallélisant plusieurs tailles de filtres dans un même bloc.

2.2.2 EfficientNet : état de l'art récent

EFFICIENTNET [6], introduit par Tan et Le en 2019, propose une approche systématique de *compound scaling* : au lieu de faire croître arbitrairement la profondeur, la largeur ou la résolution d'entrée, les auteurs montrent qu'il est optimal de *conjointement* augmenter ces trois dimensions selon un facteur unique ϕ :

$$\text{depth : } d = \alpha^\phi, \quad \text{width : } w = \beta^\phi, \quad \text{résolution : } r = \gamma^\phi \quad (2.1)$$

sous la contrainte $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$. Le bloc fondamental d'EfficientNet est le *Mobile Inverted Bottleneck* (MBConv), enrichi de *squeeze-and-excitation* [7]. Le modèle EFFICIENTNETB0 compte environ 5,3 millions de paramètres et atteint 77,3% de top-1 accuracy sur ImageNet — un excellent compromis performance/coût, particulièrement adapté au *Transfer Learning*.

2.3 Transfer Learning

L'idée fondamentale du *Transfer Learning* [11, 12] consiste à réutiliser les connaissances apprises par un modèle sur une tâche source (ici la classification d'IMAGENET, qui contient 1,28 million d'images réparties en 1 000 classes) pour résoudre une tâche cible différente mais reliée — dans notre cas, la classification binaire de pièces de fonderie. La justification théorique repose sur la nature *hiérarchique* des représentations apprises par les CNN : les premières couches détectent des concepts visuels génériques (contours, textures, gradients de couleur) qui sont *transférables* entre la quasi-totalité des tâches d'images naturelles, tandis que les dernières couches encodent des concepts plus spécifiques à la tâche source.

En pratique, le Transfer Learning s'opère selon deux stratégies, souvent combinées :

1. **Feature extraction** : le *backbone* pre-entraîné est gelé, et seule une nouvelle tête de classification est entraînée sur la tâche cible. Stratégie privilégiée lorsque le dataset cible est petit ($< 10\,000$ images).
2. **Fine-tuning** : une partie ou la totalité du backbone est dégelée et réentraîne avec un *learning rate* très faible afin d'ajuster les représentations pre-apprises sans les détruire.

2.4 Travaux relatifs à la détection de défauts en fonderie

Plusieurs études récentes ont appliqué le *deep learning* à la détection de défauts dans des contextes industriels analogues : aciers laminés [14], soudures [15], surfaces métalliques générales [16]. Ces travaux convergent sur trois enseignements :

- le Transfer Learning supplante systématiquement l’entraînement de zero pour les datasets de taille modérée ;
- les augmentations photométriques (variations d’éclairage, contraste, saturation) sont essentielles pour la généralisation à des conditions d’usine variables ;
- le *recall* sur la classe défectueuse est la métrique critique : un faux négatif à un coût asymétriquement plus élevé qu’un faux positif.

3. Procédé de fabrication et typologie des défauts

3.1 Procédé de fonderie en moule carapace

L'*impeller* (turbine ou roue de pompe) est une pièce métallique critique qui transmet l'énergie mécanique au fluide pompé. Son procédé de fabrication par fonderie en moule carapace (*shell mould casting*) se décompose en trois grandes étapes successives.

3.1.1 Étape 1 : Conception (Design of pump)

La pièce est d'abord modélisée en Conception Assistée par Ordinateur (CAO) : une roue à aubes destinée à équiper une pompe centrifuge. Cette étape détermine la géométrie finale, les tolérances dimensionnelles et la spécification de l'état de surface attendu en sortie de fonderie.

3.1.2 Étape 2 : Fabrication du moule carapace (Shell mould)

Un moule en sable aggloméré par résine thermodurcissable est confectionné autour d'un modèle en métal chauffé à environ 250°C. Le sable résine durcit au contact de la chaleur en formant une *carapace* de quelques millimètres d'épaisseur, qui sera ensuite démoulée et utilisée comme empreinte négative de la pièce à couler.

3.1.3 Étape 3 : Coulée (Shell mould casting process)

Plusieurs moules carapaces sont assemblés et serrés dans un cadre métallique. Le métal en fusion (typiquement un alliage d'aluminium, de fonte ou d'acier selon l'application) est ensuite coulé par gravité dans les empreintes, refroidit et se solidifié. Après démoulage, la pièce métallique est nettoyée, ébavurée, puis usinée si nécessaire pour atteindre les tolérances finales. La figure 3.1 illustre les principales étapes industrielles de ce procédé.



(a) Moule carapace assemblé et serré dans son cadre, prêt à recevoir le métal en fusion.

(b) Phase de coulée : le métal liquide est versé dans les moules avant solidification.

Figure 3.1 – Étapes du procédé de fonderie en moule carapace.

Lecture de la figure 3.1. La sous-figure (a) montre l'arrangement des moules carapaces serrés ensemble par un cadre extérieur : cette configuration permet de couler plusieurs pièces simultanément et d'optimiser la productivité. La sous-figure (b) illustre la phase critique de la coulée, où la qualité finale de la pièce se joue : vitesse de coulée, température du métal, vitesse de refroidissement et qualité du moule sont autant de paramètres susceptibles d'introduire des défauts internes ou de surface.

3.2 Typologie des défauts observés

Après refroidissement et démoulage, certaines pièces peuvent présenter des anomalies qui les rendent impropres à l'usage. Les défauts les plus fréquemment observés sur les impellers de fonderie sont les suivants :

- **Fissures (cracks).** Discontinuités linéaires, souvent fines, résultant de tensions internes lors du refroidissement, de chocs thermiques ou d'inclusions non métalliques. Très dangereuses car elles peuvent se propager en service sous l'effet des contraintes cycliques.
- **Deformations géométriques.** Écarts par rapport à la géométrie nominale (gauchissement d'une aube, mauvaise concentricité). Causes par un retrait différentiel pendant la solidification ou par un démoulage prématuré.
- **Bavures (flash).** Excedents de matière aux joints du moule, dus à un serrage insuffisant ou à une pression de coulée excessive.
- **Porosites de surface.** Petits pores visibles à la surface, résultant de gaz emprisonnés ou de retraits localisés.

La figure 3.2 présente un panel représentatif de ces anomalies en regard d'une pièce conforme.

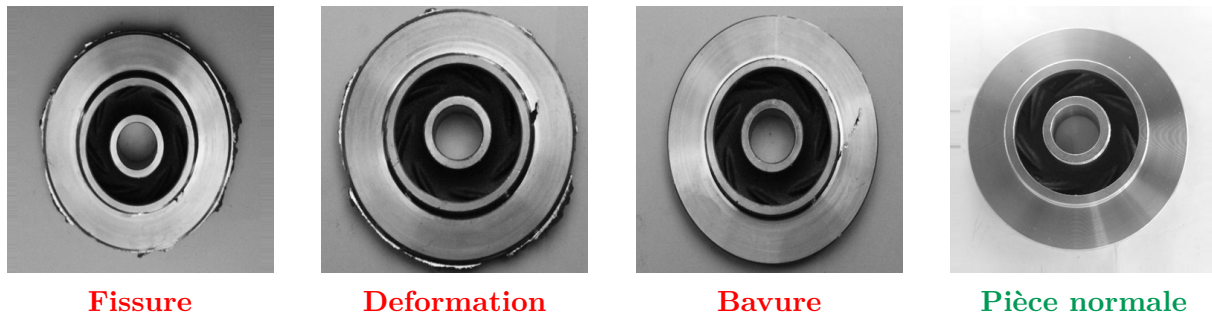


Figure 3.2 – Exemples typiques de défauts comparés à une pièce conforme.

Lecture de la figure 3.2. Cette planche illustre la diversité morphologique des anomalies. La fissure (a) est linéaire et très locale, exigeant une résolution suffisante pour être détectée. La déformation (b) modifie la géométrie globale de la pièce, ce qui en fait un défaut détectable à basse fréquence spatiale. La bavure (c) est un excédent de matière localisée au bord. Enfin, la pièce conforme (d) sert de référence visuelle à notre modèle. **Cette diversité explique pourquoi un simple traitement d'image classique échoue : les défauts n'ont ni la même échelle, ni la même texture, ni la même localisation.** Seul un modèle d'apprentissage profond, capable d'apprendre simultanément plusieurs niveaux d'abstraction visuelle, peut espérer détecter cette variété de cas.

3.3 Objectif spécifique du projet

Formulation opérationnelle

Concevoir un **classifieur binaire** prenant en entrée une image numérique d'impeller et produisant en sortie une étiquette $\hat{y} \in \{\text{def_front}, \text{ok_front}\}$ accompagnée d'un score de confiance $p \in [0, 1]$, avec :

- un *recall* sur la classe **def_front** le plus élevé possible (priorité industrielle absolue) ;
- une *précision* suffisante pour éviter les fausses alertes (coûts de production) ;
- un temps d'inférence compatible avec la cadence de la ligne (< 100 ms par pièce).

4. Acquisition et organisation des données

4.1 Choix du jeu de données

La construction d'un dataset spécifique aux impellers de fonderie aurait nécessité une campagne d'acquisition en usine, hors de portée dans le cadre de ce projet. Nous nous tournons donc vers un jeu de données public reconnu par la communauté : ***Casting Defect Detection*** [23], créé par Ravirajsinh Dabhi et publié sur Kaggle. Ce dataset présente plusieurs avantages :

- il porte spécifiquement sur des *pièces de fonderie* (et non sur un domaine générique) ;
- les images sont prises dans des conditions *réalistes* d'éclairage et d'angle de vue ;
- il distingue explicitement deux classes (`def_front` et `ok_front`) représentant un cas typique de classification binaire industrielle ;
- il propose deux versions distinctes : une version *originale* d'environ 1 300 images, et une version *augmentée* de $\sim 6\,600$ images publiée par le même auteur. Dans ce chapitre, nous décrivons la **version originale**, qui sert de référence pour les expérimentations MLP et CNN. La version augmentée sera utilisée plus loin (chapitre 6, section Transfer Learning) pour tirer pleinement parti des capacités d'EFFICIENTNETB0.

4.2 Description du dataset original

Le dataset *Casting Defect Detection* (version originale) contient des images en couleur prises à la vue frontale (*front*) de pièces de fonderie. Il est organisé en deux classes binaires :

- `def_front` : pièces présentant un défaut visible (fissure, déformation, bavure, porosité) ;
- `ok_front` : pièces conformes aux spécifications.

La figure 4.1 présente un aperçu visuel des images du dataset.

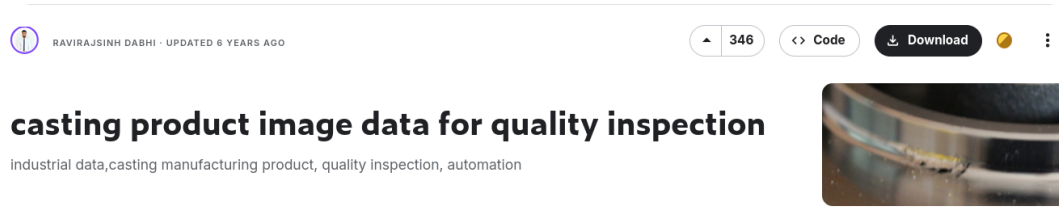


Figure 4.1 – Aperçu visuel du dataset *Casting Defect Detection* (Kaggle, version originale).

Lecture de la figure 4.1. On observe que les images partagent une mise en cadre standardisée : la pièce est centrée, prise sous un éclairage relativement uniforme et à une distance constante. Cette homogénéité est un atout pour l'apprentissage, mais elle constitue aussi une *limitation* : un modèle entraîné sur ces images ne généralisera pas nécessairement à des conditions d'usine différentes (angles obliques, éclairages non contrôlés). Cette considération sera reprise dans la discussion finale.

4.3 Répartition train / validation / test

Le dataset est partitionné en trois sous-ensembles disjoints selon la stratégie classique 80 / 10 / 10 :

Ensemble	Proportion	Rôle
Train	80 %	Apprentissage des paramètres du modèle
Validation	10 %	Réglage des hyperparamètres / <i>early stopping</i>
Test	10 %	Évaluation finale (jamais vu durant l'apprentissage)

Table 4.1 – Répartition train / validation / test du dataset.

Justification. Cette partition présente trois propriétés essentielles :

1. **Disjonction stricte** : aucune image n'appartient simultanément à deux ensembles. Cela garantit que les performances mesurées sur le test reflètent une vraie capacité de *généralisation*.
2. **Stratification** : la proportion des classes est préservée dans chaque sous-ensemble, évitant tout biais d'échantillonnage.
3. **Reproductibilité** : la partition est fixée par l'auteur du dataset, ce qui permet la comparaison directe entre études utilisant ce même jeu de données.

4.4 Volumes et déséquilibre des classes

Le tableau 4.2 récapitule les volumes par sous-ensemble et par classe pour la **version originale** du dataset, qui totalise environ 1 300 **images** réparties comme suit :

Ensemble	def_front	ok_front	Total
Train	546	363	909
Validation	117	77	194
Test	118	79	197
Total	781	519	1 300

Table 4.2 – Volumes du dataset original *Casting Defect Detection* par split et par classe.

La distribution observée est légèrement déséquilibrée, avec environ **60 % de pièces défectueuses** (`def_front`) contre **40 % de pièces conformes** (`ok_front`). La figure 4.2 illustre visuellement cette répartition.

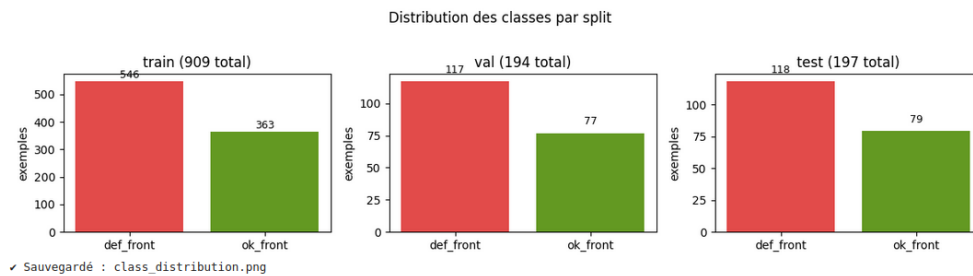


Figure 4.2 – Distribution des classes par sous-ensemble (train / validation / test).

Lecture de la figure 4.2. Les trois histogrammes confirment que la proportion entre les deux classes est *conservée* d'un sous-ensemble à l'autre, témoignant d'un échantillonnage *stratifié* bien réalisé. La barre rouge (`def_front`) domine systématiquement la barre verte (`ok_front`), avec un ratio quasi constant d'environ 60 %/40 %. Cette stabilité est importante car elle garantit que la performance mesurée sur la validation et le test reflète la même distribution que celle vue à l'entraînement.

Pourquoi le déséquilibre est-il un problème ?

En l'absence de correction, un modèle entraîné sur un dataset déséquilibré tend à apprendre la *distribution prior* des classes plutôt que la séparation réelle entre celles-ci. Dans notre cas, un modèle « naïf » pourrait prédire systématiquement `def_front` et atteindre $\sim 60\%$ d'*accuracy* sans rien avoir appris du contenu des images ! Ce piège est fondamental et doit être traité explicitement par un mécanisme de **pondération des classes** (*class weights* en TensorFlow ou `WeightedRandomSampler` en PyTorch), qui sera détaillé dans le chapitre suivant.

4.5 Vers la version augmentée pour le Transfer Learning

Le dataset original ($\sim 1\,300$ images) est suffisant pour les modèles MLP et CNN développés dans les sections suivantes, mais il devient *trop limité* pour exploiter pleinement la capacité d'un grand réseau pré-entraîné comme EFFICIENTNETB0. Pour cette raison, nous utiliserons dans le chapitre dédié au Transfer Learning (chapitre 6) la **version augmentée** du même auteur, qui porte le nombre d'images d'entraînement à environ 6 600 et l'ensemble de test à 715 images. Cette version intègre des transformations d'augmentation spécifiquement adaptées aux contraintes industrielles (variations d'éclairage, angles, textures de surface), ce qui en fait un choix pragmatique pour notre étude de Transfer Learning.

5. Prétraitement des images

5.1 Philosophie générale du pipeline

La phase de prétraitement est l'une des plus déterminantes en apprentissage profond appliqué à la vision : elle conditionne directement la *capacité de généralisation* du modèle final. Notre pipeline est conçu autour de quatre principes directeurs :

Reproductibilité. Toutes les sources d'aléatoire (Python, NumPy, frameworks) sont initialisées par une graine globale unique `SEED = 42`. Une expérience peut donc être reproduite à l'identique à partir du même code et des mêmes données.

Modularité. Les transformations sont implémentées comme des fonctions indépendantes, recombinaisons, réutilisables.

Portabilité. Le pipeline est implémenté **en double version** (TensorFlow et PyTorch), validant ainsi la robustesse de la conception.

Performance. Utilisation systématique du parallélisme (`tf.data.AUTOTUNE` ou `num_workers`) et du *prefetching* pour saturer le Graphics Processing Unit (GPU).

5.2 Vue d'ensemble du pipeline TensorFlow (10 sections)

L'implémentation TensorFlow est structurée en **dix sections fonctionnelles** successives, illustrées par le bandeau de la figure 5.1.

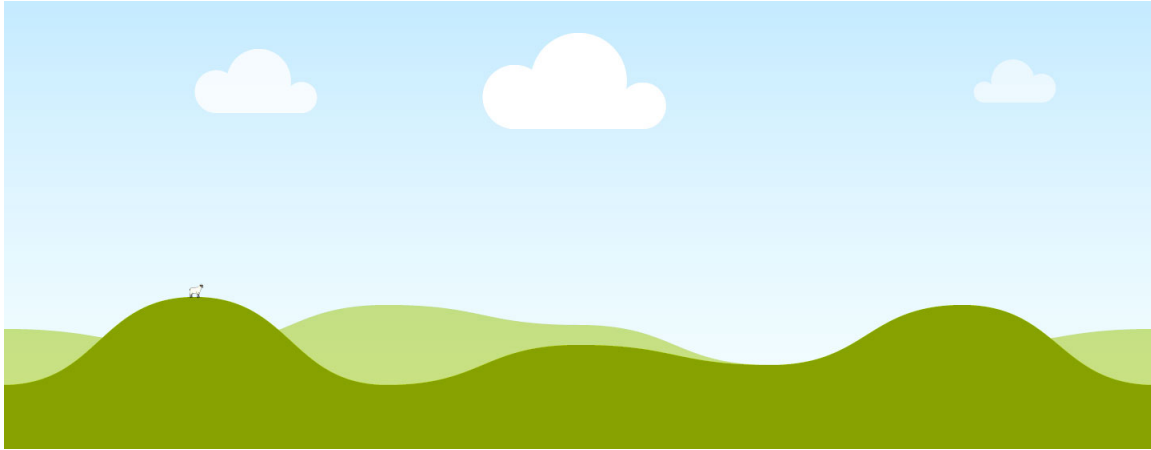


Figure 5.1 – Bandeau de présentation du pipeline de prétraitement TensorFlow.

Lecture de la figure 5.1. Ce bandeau, utilisé comme séparateur visuel dans le notebook, marqué l'entrée dans la partie TENSORFLOW du pipeline. Il symbolise la chaîne de transformations que chaque image traversera, depuis sa lecture brute sur le disque jusqu'à son arrivée comme tenseur normalisé dans le modèle.

Les dix sections de cette chaîne sont les suivantes :

1. **Reproductibilité** : SEED = 42 (random, NumPy, TensorFlow).
2. **Détection GPU/CPU** : `tf.config.list_physical_devices()`.
3. **Color Balancing** : White Balance, Histogram Equalization, Channel Shuffle.
4. **Normalisation ImageNet** : $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$.
5. **Pipeline Train** : crop 224×224 , flips, rotations, jitter.
6. **Pipeline Eval** : *center crop* déterministe + white balance.
7. **Chargement** : `image_dataset_from_directory` pour train/val/test.
8. **Class Weights** : équivalent du `WeightedRandomSampler` de PyTorch.
9. **Pipelines tf.data** : `.map()` $\rightarrow$ `.batch()` $\rightarrow$ `.prefetch(AUTOTUNE)`.
10. **Configuration** : export JSON, sanity check, visualisation.

5.2.1 Reproductibilité et détection du *device*

Pour garantir la reproductibilité, toutes les sources d'aléatoire sont fixées à une même graine :

```

1 SEED = 42
2 random.seed(SEED)
3 np.random.seed(SEED)
4 tf.random.set_seed(SEED)
5
6 gpus = tf.config.list_physical_devices("GPU")
7 DEVICE = "GPU" if gpus else "CPU"
8 if gpus:
9     for gpu in gpus:
10         tf.config.experimental.set_memory_growth(gpu, True)

```

Listing 5.1 – Initialisation reproductible et détection automatique du device.

Commentaire. L’option `set_memory_growth` est essentielle dans les environnements partagés comme Google Colab : elle indique à TensorFlow de n’allouer la mémoire GPU qu’au fur et à mesure des besoins, plutôt que de réserver l’intégralité de la VRAM dès l’initialisation. Cela évite les erreurs `Out-Of-Memory` lors de l’exécution simultanée de plusieurs notebooks.

Hyperparamètres globaux. Le tableau 5.1 récapitule les constantes globales partagées par tous les notebooks du projet.

Constante	Valeur	Role
IMG_SIZE	224	Côté image (carrée)
BATCH_SIZE	32	Images par batch
NUM_CLASSES	2	<i>defect / ok_front</i>
MEAN	[0.485, 0.456, 0.406]	Moyenne ImageNet (R/G/B)
STD	[0.229, 0.224, 0.225]	Écart-type ImageNet (R/G/B)
AUTOTUNE	<code>tf.data.AUTOTUNE</code>	Parallelisme automatique

Table 5.1 – Constantes globales partagées par tous les notebooks.

5.2.2 Color balancing : compatibilité *graph* TensorFlow

Les images industrielles souffrent souvent de *dominantes colorées* liées aux conditions d’éclairage (lumières jaunâtres ou bleuâtres des ateliers). Pour rendre le modèle robuste à ces variations, nous mettons en place trois transformations stochastiques de *color balancing* :

Gray-World White Balance ($p = 0,5$). Sous l’hypothèse *Gray-World* (la moyenne d’une scène naturelle tend vers le gris), on normalise chaque canal par sa moyenne, puis

on appliqué un facteur d'échelle borne dans $[0,5 ; 2,0]$ pour éviter la sur-exposition. Implementée nativement avec `tf.cond`.

Histogram Equalization ($p = 0,3$). Egalise l'histogramme canal par canal via `PIL.ImageOps.equalize` encapsulée dans `tf.py_function`. Fait ressortir les défauts subtils dans les zones peu contrastées.

Channel Shuffle ($p = 0,15$). Permute aléatoirement l'ordre des canaux RGB (6 permutations possibles), pour rendre le modèle *robuste aux dérives de balance des couleurs* et l'empêcher d'apprendre des corrélations arbitraires entre teinte et classe.

```

1 means = tf.reduce_mean(image, axis=[0, 1])
2 global_mean = tf.reduce_mean(means)
3 scale = global_mean / means # (3,)
4 scale = tf.clip_by_value(scale, 0.5, 2.0)
5 return tf.clip_by_value(image * scale, 0., 1.)

```

Listing 5.2 – Implementation du Gray-World White Balance.

Commentaire. Le clamp $[0.5, 2.0]$ est crucial : sans lui, des images fortement saturées dans un canal pourraient produire des facteurs d'échelle extrêmes qui détruiraient l'information. Cette borne pragmatique est fixée empiriquement.

5.2.3 Normalisation ImageNet

La normalisation ImageNet aligne la distribution statistique de nos images sur celle attendue par les backbones pre-entraînés :

$$\hat{x} = \frac{x - \mu}{\sigma}, \quad \mu = \begin{pmatrix} 0.485 \\ 0.456 \\ 0.406 \end{pmatrix}, \quad \sigma = \begin{pmatrix} 0.229 \\ 0.224 \\ 0.225 \end{pmatrix} \quad (5.1)$$

L'équation (5.1) centre chaque canal sur la moyenne ImageNet et le réduit par son écart-type. Cette étape est *indispensable* pour le *Transfer Learning* : les filtres pre-entraînés de EFFICIENTNETB0 ont été optimisés sur des entrées ainsi normalisées, et tout écart de distribution entraînerait une chute drastique de performance.

5.2.4 Comparaison train vs eval

Une règle absolue en apprentissage profond : **l'augmentation de données ne s'applique qu'à l'entraînement**. Le tableau 5.2 compare les transformations appliquées dans chaque mode.

Operation	preprocess_train	preprocess_eval
Resize	256×256	256×256
Crop	random 224×224	center crop 224×224
Flips	H + V	—
Rotation	$\pm 180^\circ$ (tfa)	—
Color balance	stochastique	déterministe ($p = 1$)
Color jitter	brightness/contrast/sat/hue	—
Grayscale	$p = 0,1$	—
Normalisation	ImageNet	ImageNet

Table 5.2 – Comparaison des transformations *train* vs *eval*.

Pourquoi cette dissymetrie ? En entraînement, on cherche à *augmenter virtuellement* la taille du dataset par des transformations stochastiques, ce qui force le modèle à apprendre des représentations invariantes. En évaluation, au contraire, on veut une mesure *déterministe et reproductible* des performances : appliquer des transformations aléatoires biaiserait la mesure. Le seul *color balancing* eval appliqué est le White Balance avec $p = 1$, déterministe par construction.

5.2.5 Chargement et pondération des classes

```

1 def _load_raw(directory, shuffle):
2     return tf.keras.utils.image_dataset_from_directory(
3         directory, labels="inferred", label_mode="int",
4         color_mode="rgb", batch_size=None,
5         image_size=(256, 256), shuffle=shuffle, seed=SEED,
6     )
7
8 raw_train = _load_raw(TRAIN_DIR, shuffle=True)
9 raw_val   = _load_raw(VAL_DIR,   shuffle=False)
10 raw_test  = _load_raw(TEST_DIR,  shuffle=False)
11
12 all_labels = np.array([lbl.numpy() for _, lbl in raw_train])
13 class_counts = np.bincount(all_labels, minlength=NUM_CLASSES)
14 total        = len(all_labels)
15 class_weights = {
16     i: float(total) / (NUM_CLASSES * int(count))
17     for i, count in enumerate(class_counts)
18 }

```

Listing 5.3 – Chargement `tf.data` et calcul des poids de classes.

La pondération suit la formule classique *inverse frequency weighting* :

$$w_i = \frac{N_{\text{total}}}{N_{\text{classes}} \cdot n_i} \quad (5.2)$$

Cette formule donne un poids supérieur à la classe minoritaire (`ok_front`) afin que le modèle consacre proportionnellement plus d'attention aux échantillons rares lors du calcul de la fonction de perte.

5.2.6 Composition finale des datasets

```

1 train_dataset = (raw_train
2     .map(preprocess_train, num_parallel_calls=AUTOTUNE)
3     .batch(BATCH_SIZE, drop_remainder=True)
4     .prefetch(AUTOTUNE))
5
6 val_dataset = (raw_val
7     .map(preprocess_eval, num_parallel_calls=AUTOTUNE)
8     .batch(BATCH_SIZE).prefetch(AUTOTUNE))
9
10 test_dataset = (raw_test
11     .map(preprocess_eval, num_parallel_calls=AUTOTUNE)
12     .batch(BATCH_SIZE).prefetch(AUTOTUNE))

```

Listing 5.4 – Composition finale : `.map()` -> `.batch()` -> `.prefetch()`.

Commentaire. L'ordre `.map()` → `.batch()` → `.prefetch()` est crucial. `.map()` appliqué la transformation à chaque image *individuellement* en parallèle, `.batch()` regroupe ensuite les images traitées en batches, et `.prefetch(AUTOTUNE)` permet au CPU de préparer le prochain batch pendant que le GPU traite l'actuel — réalisant ainsi un pipelining qui maximise l'utilisation des deux unités de calcul.

5.2.7 Configuration centralisée et utilitaires

```

1 CONFIG = {
2     "img_size"      : 224,
3     "batch_size"    : 32,
4     "num_classes"   : 2,
5     "class_names"   : ["defect", "ok_front"],
6     "mean"          : [0.485, 0.456, 0.406],
7     "std"           : [0.229, 0.224, 0.225],
8     "class_weights" : {"0": 1.12, "1": 0.90},
9     "device"        : "GPU"
10 }

```

Listing 5.5 – Export centralise de la configuration au format JSON.

Quatre utilitaires complètent le module `preprocessing_tf` :

- `sanity_check()` — affiche *shape*, min/max/mean d'un batch.
- `show_batch(ds, n=8)` — visualise *n* images avec leur label.
- `plot_class_distribution()` — histogramme *defect* vs *ok_front*.
- `compute_mean_std(ds, n=20)` — moyenne et écart-type empiriques.

La figure 5.2 regroupe les sorties typiques de ces quatre utilitaires.



Figure 5.2 – Sorties des utilitaires de contrôle du pipeline.

Lecture de la figure 5.2. La sous-figure (a) confirme la cohérence des shapes du batch ($32 \times 224 \times 224 \times 3$) ainsi que les statistiques pixel après normalisation (min ≈ -2.1 , max ≈ 2.6 , $\mu \approx 0.3$) — valeurs cohérentes avec la normalisation ImageNet, ce qui valide la chaîne de prétraitement. La sous-figure (b) présente un batch de 8 images après prétraitement complet : on y voit des pièces dans des orientations variées, traduisant l'effet des rotations et des flips. La sous-figure (c) confirme visuellement le déséquilibre annoncé des classes : la barre rouge (`def_front`) est systématiquement supérieure à la verte (`ok_front`) dans les trois splits, ce qui justifie à posteriori le mécanisme de *class weighting* mis en place.

5.3 Pipeline équivalent en PyTorch

Le pipeline PYTORCH reprend les mêmes étapes que la version TensorFlow, mais s'appuie sur les abstractions natives de `torchvision.transforms` et un `DataLoader` avec `WeightedRandomSampler`.

5.3.1 Reproductibilité et device

```

1 SEED = 42
2 random.seed(SEED); np.random.seed(SEED)
3 torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
4 torch.backends.cudnn.deterministic = True
5 torch.backends.cudnn.benchmark = False
6

```

```
7 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Listing 5.6 – Reproductibilité et détection automatique du device en PyTorch.

Commentaire. Les options `cudnn.deterministic = True` et `cudnn.benchmark = False` forcent CUDA à utiliser des algorithmes déterministes, au prix d’une légère perte de performance. C’est un *compromis assumé* en faveur de la reproductibilité, particulièrement important en contexte académique ou industriel où les résultats doivent pouvoir être reproduits à l’identique.

5.3.2 Color balancing personnalisée

PyTorch ne propose pas nativement les transformations de *color balancing* dont nous avons besoin ; nous les implementons donc comme des classes héritant de `torch.nn.Module` :

- **AutoWhiteBalance** ($p = 0,5$) — hypothèse *Gray-World* pour corriger les dominantes colorées des caméras industrielles.
- **HistogramEqualization** ($p = 0,3$) — fait ressortir les défauts subtils dans les zones peu contrastées.
- **ChannelShuffle** ($p = 0,15$) — permute les canaux RGB pour rendre le modèle indépendant de la teinte de surface.

5.3.3 Pipeline complet d’augmentations

Les transformations appliquées au *train* se composent comme suit :

1. `Resize(256, 256)` — uniformisation de la taille d’entrée.
2. `RandomCrop(224)` — recadrage aléatoire 224×224 pour introduire de la variabilité spatiale.
3. `RandomHorizontalFlip(0.5)` et `RandomVerticalFlip(0.5)` — les images industrielles n’ont pas d’orientation privilégiée.
4. `RandomRotation(180)` — rotations jusqu’à $\pm 180^\circ$: invariance rotationnelle complète.
5. `AutoWhiteBalance(0.5)`, `HistogramEqualization(0.3)`, `ChannelShuffle(0.15)` — robustesse couleur.

6. `ColorJitter(0.2, 0.3, 0.1, 0.02)` — variations *brightness, contrast, saturation, hue*.
7. `RandomAffine(translate=(0.05, 0.05))` — translations $\pm 5\%$: robustesse aux decalages de cadrage.
8. `RandomGrayscale(0.1)` — 10% des images converties en niveaux de gris.
9. `GaussianBlur(3)` — flou léger : robustesse au bruit de capteur.
10. `ToTensor()` et `Normalize( $\mu, \sigma$ )` (ImageNet).

5.3.4 Gestion du déséquilibre des classes

Le dataset présente un léger déséquilibre ($\sim 65\%$ `def_front` vs $\sim 35\%$ `ok_front`). Sans correction, le modèle apprendrait à toujours prédire la classe majoritaire. Nous utilisons un `WeightedRandomSampler` :

```

1 class_counts    = torch.bincount(targets)
2 class_weights   = 1.0 / class_counts
3 sample_weights  = class_weights[targets]
4 sampler = WeightedRandomSampler(
5     sample_weights, num_samples=len(sample_weights),
6     replacement=True
7 )

```

Listing 5.7 – Pondération inverse-fréquence des échantillons.

Commentaire. L’option `replacement=True` est essentielle : sans elle, les échantillons de la classe minoritaire seraient épuisés avant ceux de la classe majoritaire. Avec `replacement`, chaque époque tire N échantillons au sort selon la distribution pondérée, garantissant une exposition équilibrée du modèle aux deux classes.

5.3.5 DataLoaders et utilitaires

Le tableau 5.3 resume la configuration des trois `DataLoaders`.

Loader	batch_size	shuffle	Sampler	pin_memory
<code>train_loader</code>	32	—	<code>WeightedRandom</code>	si CUDA
<code>val_loader</code>	32	False	—	si CUDA
<code>test_loader</code>	32	False	—	si CUDA

Table 5.3 – Configuration des trois `DataLoaders` PyTorch.

Pourquoi `pin_memory=True` ? L'option `pin_memory` alloue les tenseurs dans une zone mémoire CPU « pinnée », accélérant le transfert vers le GPU via DMA asynchrone. Le gain peut atteindre 30% sur les transferts CPU-GPU, ce qui est significatif lorsque le bottleneck du training est le chargement des données.

6. Modélisation et expérimentations

Dans ce chapitre, nous développons et comparons **trois architectures de classification** suivant une démarche progressive : (i) un Perceptron Multi-Couches (MLP) servant de baseline, (ii) un Réseau de Neurones Convolutif (CNN) entraîné de zéro, et (iii) un modèle de *Transfer Learning* fondé sur EFFICIENTNETB0. Cette démarche permet de mettre en évidence, de manière empirique et pédagogique, l'apport spécifique de chaque famille architecturale.

6.1 Modèle 1 : Perceptron Multi-Couches (MLP)

6.1.1 Motivation

Le MLP constitue le modèle de neurones le plus élémentaire capable d'apprentissage supervisé non-linéaire. Il sert ici de **baseline** : à la fois comme référence de performance *minimale* et comme prétexte pédagogique pour mettre en évidence les limites des réseaux *denses* face aux données structurées comme les images.

6.1.2 Representation des données : le Flatten

Les images d'entrée sont des tenseurs $224 \times 224 \times 3$ (hauteur $\times$ largeur $\times$ canaux). Or, un MLP ne sait traiter que des *vecteurs unidimensionnels* : il faut donc *aplatir* l'image, opération réalisée par la couche **Flatten** :

$$(224 \times 224 \times 3) \longrightarrow \mathbb{R}^{150\,528} \quad (6.1)$$

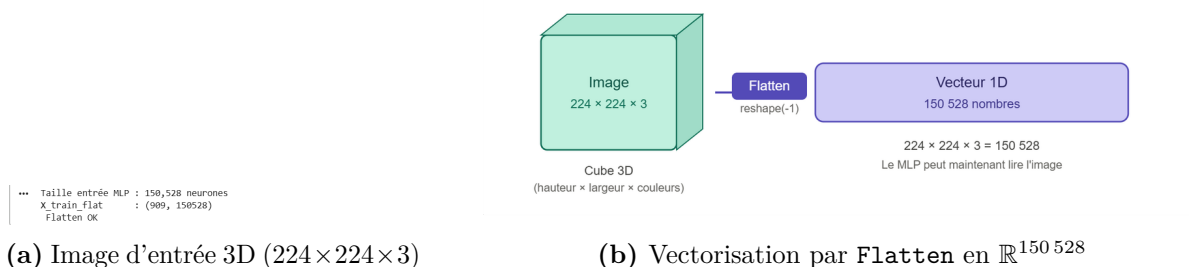


Figure 6.1 – Aplatissement (*Flatten*) d'une image en vecteur 1D.

Lecture de la figure 6.1. La sous-figure (a) représente l'image dans sa structuré naturelle 3D, ou la *proximité spatiale* entre pixels traduit une réalité physique : deux pixels

voisins horizontalement ou verticalement appartiennent typiquement à la même region (même aube, même bord). La sous-figure (b) illustre la transformation **Flatten**, qui *linéarise* ces 150 528 valeurs sans aucune notion d'adjacence. **C'est la limite fondamentale du MLP appliqué aux images** : deux pixels voisins en 2D peuvent se retrouver très éloignés dans le vecteur 1D, ce qui empêche le réseau de tirer parti de la structuré spatiale.

6.1.3 Architecture du MLP

Architecture du MLP

Entrée : 150 528 neurones

→ Dense(512) + BatchNorm + ReLU + Dropout(0.4)

→ Dense(256) + BatchNorm + ReLU + Dropout(0.4)

→ Dense(128) + BatchNorm + ReLU + Dropout(0.3)

→ Dense(2) + **Softmax** (→ `def_front` / `ok_front`)

La figure 6.2 représente schématiquement cette architecture.

Model: "MLP"		
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	77,070,848
batch_normalization	(None, 512)	2,048
activation (ReLU)	(None, 512)	0
dropout (Dropout 0.4)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1	(None, 256)	1,024
activation_1 + dropout_1 (0.4)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
batch_normalization_2	(None, 128)	512
activation_2 + dropout_2 (0.3)	(None, 128)	0
dense_3 (Dense + Softmax)	(None, 2)	258
Total params: 77,238,914 (294.64 MB)		
Trainable params: 77,237,122 (294.64 MB)		
Non-trainable: 1,792 (7.00 KB)		

Figure 6.2 – Schéma global de l'architecture MLP.

Lecture de la figure 6.2. Le réseau enchaîne trois blocs denses dont la dimension décroît progressivement (512 → 256 → 128). Cette décroissance, classique en architecture neuronale, joue le rôle d'un *compresseur d'information* : le réseau doit progressivement *filtrer* l'information pertinente parmi les 150 528 features brutes. La sortie est une couche dense de 2 neurones suivie d'une fonction *Softmax* qui produit une distribution de probabilité sur les deux classes.

6.1.4 Mécanismes de régularisation

Trois mécanismes de régularisation sont intégrés pour limiter le sur-apprentissage :

- **Batch Normalization** [8] : normalise les activations intermédiaires sur le batch, stabilisant et accélérant l'apprentissage. Mathématiquement, pour une activation x d'un mini-batch :

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma\hat{x} + \beta \quad (6.2)$$

ou γ, β sont des paramètres appris.

- **Dropout** [9] : a chaque iteration d'entraînement, une fraction p des neurones est aléatoirement désactivée. Cela force le réseau à apprendre des représentations *redondantes* et l'empêche de trop dépendre d'un sous-ensemble de neurones.
- **Early Stopping (patience = 7)** : arrêt automatique de l'entraînement si la perte de validation cesse de diminuer pendant 7 époques consécutives. Sauvegarde le meilleur modèle observé.
- **ReduceLROnPlateau** : divise le *learning rate* par 2 si la perte stagne, permettant un *fine-tuning* progressif.

6.1.5 Résultats du MLP

Le modèle est entraîné pendant un maximum de 30 époques avec optimiseur Adam (LR initial = 1e-3) et perte *categorical crossentropy*. Les courbes d'apprentissage et la matrice de confusion finale sont présentées figure 6.3.

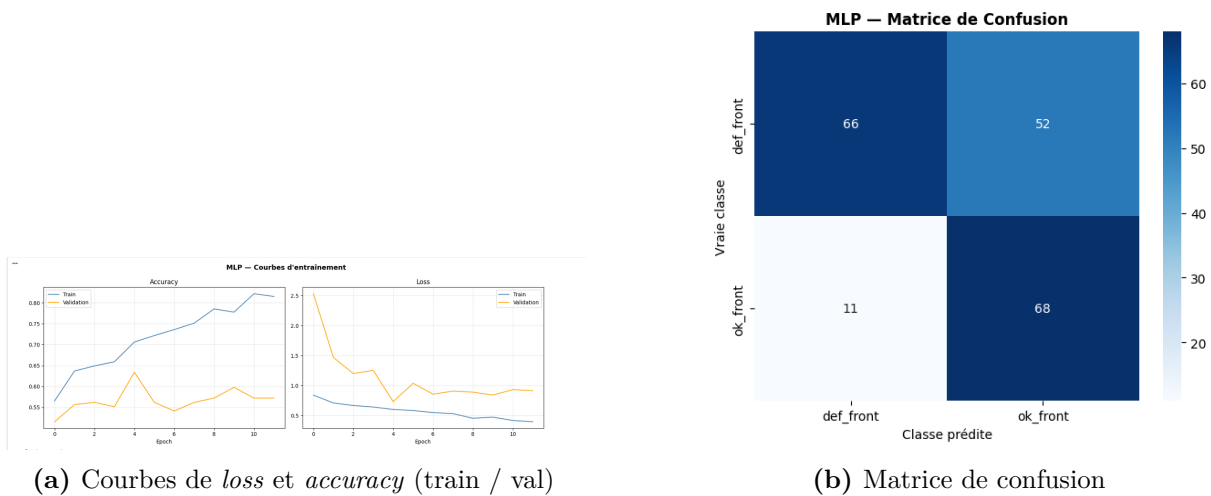


Figure 6.3 – Performance du MLP sur le jeu de test.

Lecture de la figure 6.3. La sous-figure (a) montre que l'*accuracy* de train et de validation évoluent de manière proche, plafonnant rapidement autour de $\sim 70\%$. La perte (*loss*) décroît puis se stabilise. L'absence d'écart marqué entre train et validation indique **l'absence de sur-apprentissage** : le problème du MLP n'est pas qu'il apprend par cœur ses données, mais qu'il n'est *intrinsequement pas capable* d'apprendre une fonction de classification suffisamment expressive sur ce type d'entrée. La sous-figure (b) confirme cette analyse : la matrice de confusion montre des erreurs symétriquement réparties sur les deux classes (52 `def_front` mal classées, 11 `ok_front` mal classées), traduisant une difficulté structurelle plutôt qu'un biais.

Le tableau 6.1 synthétise les métriques finales sur le test.

Métrique	Valeur
Test Loss	0,7587
Test Accuracy	68,02 %
Precision	74,07 %
Recall	68,02 %
F1-Score	67,95 %
Epochs entraînés	12/30 (early stopping déclenche)

Table 6.1 – Résultats du MLP sur l'ensemble de test.

6.1.6 Analyse critique du MLP

Points forts. Convergence rapide (12 époques), modèle simple à comprendre et à entraîner, baseline honnête pour quantifier l'apport des architectures suivantes.

Limites. La couche `Flatten` **détruit la structuré spatiale** : les pixels voisins perdent leur lien, le modèle ne peut pas apprendre de *textures* ni de *contours*. De plus, le nombre colossal de paramètres dans la première couche dense ($150\,528 \times 512 \approx 77$ millions!) est totalement disproportionné par rapport à la taille du dataset, augmentant le risque de sur-apprentissage et alourdissant inutilement le modèle.

⇒ Ces limitations motivent le passage à un modèle exploitant explicitement la structuré 2D de l'image : le **Réseau de Neurones Convolutif (CNN)**.

6.2 Modèle 2 : CNN entraîne de zero

6.2.1 Principe et fondements théoriques

Un **Réseau de Neurones Convolutif (CNN)** [1] est une architecture conçue spécifiquement pour exploiter la structuré spatiale des images. Il repose sur trois opérations fondamentales :

1. **La convolution** : un filtre (ou *kernel*) de taille réduite (typiquement 3×3) glisse sur l'image et calcule un produit scalaire local. Le même filtre est appliqué partout (*poids partagés*), ce qui réduit drastiquement le nombre de paramètres et confère au réseau une *invariance par translation*.
2. **Le pooling** : réduction spatiale (*max pooling* ou *average pooling*) qui résume une région locale en un unique pixel, augmentant le champ réceptif et réduisant la dimension.
3. **Les non-linearités** : typiquement ReLU (Rectified Linear Unit), qui introduit la capacité de modéliser des fonctions non-linéaires complexes.

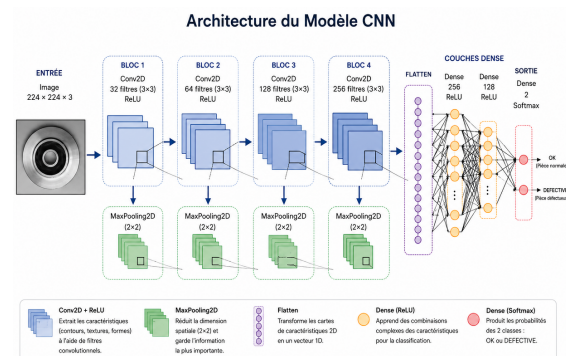
L'empilement de ces opérations construit une *hiérarchie de représentations* : les premières couches détectent des contours et des couleurs, les couches intermédiaires des textures et des motifs, les couches profondes des concepts sémantiques.

6.2.2 Architecture du CNN propose

Notre CNN enchaîne plusieurs blocs convolutifs, chacun composé d'une couche **Conv2D**, suivie de **BatchNormalization**, d'une activation **ReLU** et d'une couche **MaxPooling2D**. La figure 6.4 illustre l'architecture globale.

```
model_cnn = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    # Bloc 1
    layers.Conv2D(32, (3,3), activation='relu',padding='same'),
    layers.MaxPooling2D((2,2)),
    #Bloc 2
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Bloc 3
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Bloc 4
    layers.Conv2D(256, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Flatten
    layers.Flatten(),
    # Dense layers
    layers.Dense(256, activation='relu'),
    layers.Dense(128, activation='relu'),
    # Output
    layers.Dense(2, activation='softmax')])
```

(a) Bloc convolutif élémentaire



(b) Vue d'ensemble du CNN

Figure 6.4 – Architecture du CNN entraîné de zéro.

Lecture de la figure 6.4. La sous-figure (a) présente la *cellule élémentaire* répétée dans le réseau : convolution, batch norm, ReLU, pooling. C'est l'unité atomique d'extraction de caractéristiques. La sous-figure (b) montre l'agencement global : plusieurs blocs convolutifs en série, dont la profondeur (nombre de filtres) augmente progressivement, suivis d'un *Global Average Pooling* et d'une tête de classification dense. Cette architecture pyramidale est typique des CNN et reflète la hiérarchie d'abstraction visuelle évoquée plus haut.

6.2.3 Compilation et régularisation

Le modèle est compilé avec :

- **Optimiseur** : Adam [10] avec `learning_rate=1e-3`.
- **Fonction de perte** : *categorical_crossentropy* (formulation pour 2 classes one-hot).
- **Métriques** : accuracy.

Plusieurs mécanismes de régularisation sont intégrés :

```
model_cnn.compile(optimizer="adam",  
                  loss="sparse_categorical_crossentropy",  
                  metrics=["accuracy"])
```

Figure 6.5 – Compilation et configuration du CNN avec callbacks de régularisation.

Lecture de la figure 6.5. La compilation Keras révèle trois mécanismes de régularisation simultanés : l'*Early Stopping* (arrêt si la `val_loss` ne s'améliore plus), la *Batch Normalization* (déjà décrite, intégrée dans chaque bloc convolutif) et le *Dropout* (desactivation aléatoire de neurones). La combinaison de ces trois mécanismes correspond aux meilleures pratiques actuelles pour entraîner un CNN sur un dataset de taille modérée.

6.2.4 Entraînement

Le CNN est entraîné pendant 30 époques maximum, avec batch de taille 32. La figure 6.6 montre la boucle d'entraînement et un extrait des logs Keras.

```

model_cnn_reg = models.Sequential([
    #Input
    layers.Input(shape=(224, 224, 3)),
    #Block 1
    layers.Conv2D(32, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 2
    layers.Conv2D(64, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 3
    layers.Conv2D(128, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 4
    layers.Conv2D(256, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Flatten
    layers.Flatten(),
    #Dense layers
    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.5),
    layers.Dense(128),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.5),
    # Output
    layers.Dense(2, activation='softmax')])

```

(a) Boucle d'entraînement `model.fit()`

```

early_stop_cnn = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history_cnn_reg = model_cnn_reg.fit(
    train_loader,
    validation_data=val_loader,
    epochs=20,
    callbacks=[early_stop_cnn],
    verbose=1
)

```

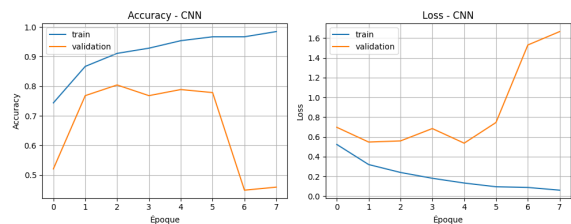
(b) Extrait des logs Keras

Figure 6.6 – Entraînement du CNN sur 30 epoques.

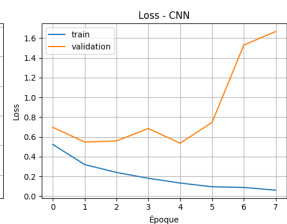
Lecture de la figure 6.6. La sous-figure (a) montre l'appel canonique `model.fit()` avec ses paramètres essentiels : dataset d'entraînement, dataset de validation, nombre d'epoques, callbacks. La sous-figure (b) présente un extrait des logs : on y voit les progrès époque par époque, avec accuracy de train et accuracy de validation. Ces logs sont essentiels pour le diagnostic en temps réel du comportement du modèle.

6.2.5 Évaluation et diagnostic d'overfitting

Les courbes d'apprentissage du CNN sans data augmentation revelent un comportement caractéristique illustre figure 6.7.



(a) Accuracy train (bleue) vs validation (orange)



```

test_loss_cnn_reg, test_acc_cnn_reg = model_cnn_reg.evaluate(
    test_loader,
    verbose=0
)

print("Test Loss:", test_loss_cnn_reg)
print("Test Accuracy:", test_acc_cnn_reg)

Test Loss: 0.6228603720664978
Test Accuracy: 0.8020304441452026

```

(b) Loss train (bleue) vs validation (orange)

Figure 6.7 – Courbes d'entraînement du CNN avant data augmentation : signe clair de sur-apprentissage.

Lecture de la figure 6.7 : diagnostic d'overfitting.

- *Accuracy de train* : monte rapidement jusqu'à $\sim 99\%$ $\rightarrow$ le modèle apprend très bien les données d'entraînement.
- *Accuracy de validation* : monte d'abord en parallèle, puis **plafonne** et commence même à **décroître** dans les dernières epoques.

- *Loss de train* : décroît regulierement.
- *Loss de validation* : décroît d'abord, puis **remonte** a partir de l'epoque ~ 15 .

⇒ **Diagnostic clair de sur-apprentissage (overfitting)** : le modèle mémorise les données d'entraînement plutôt que d'apprendre des règles generalisables. La divergence entre courbes de train et de validation est le marqueur classique de ce phénomène.

6.2.6 Regularisation par augmentation des données

Pour combattre l'overfitting, nous renforçons la *data augmentation* en ajoutant des transformations supplémentaires (rotations, flips, jitter colore, zoom). Ces transformations *augmentent virtuellement* la taille du dataset et forcent le modèle à apprendre des représentations invariantes à ces variations. Les résultats après augmentation sont présents figure 6.8.

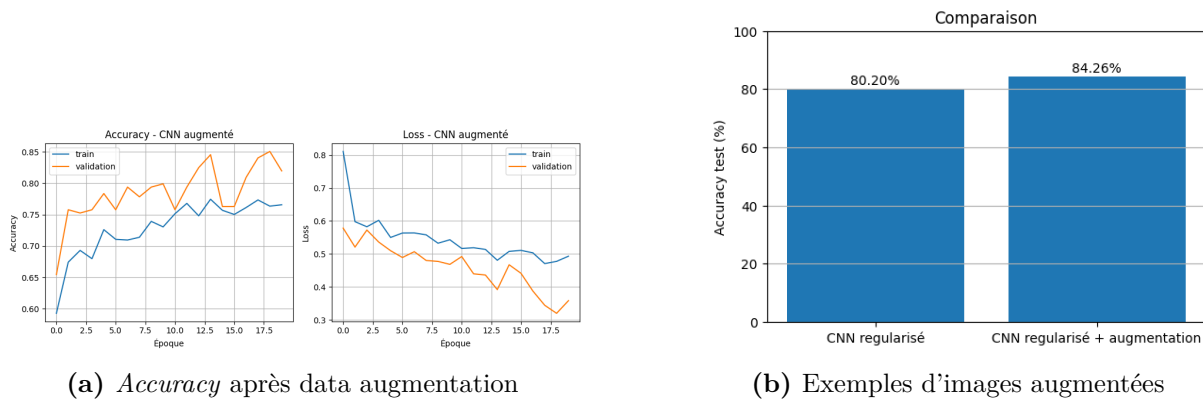


Figure 6.8 – Effet de la *data augmentation* sur l'apprentissage.

Lecture de la figure 6.8. La sous-figure (a) montre des courbes *accuracy* train et validation désormais beaucoup plus proches l'une de l'autre, avec une validation qui suit la train sans plonger : **l'overfitting est largement resorbé**. La sous-figure (b) illustre quelques transformations appliquées au même échantillon : rotations diverses, flips, variations de luminosité. Ces « copies légèrement perturbées » multiplient virtuellement la diversité des exemples vus par le réseau.

6.2.7 Conclusion intermédiaire sur le CNN

Bilan du CNN

La data augmentation corrige efficacement l'overfitting et fournit un modèle plus robuste. Cependant, atteindre 99% de précision avec un CNN *entraîne de zero* nécessiterait beaucoup plus de données ($> 10^5$ images typiquement). Sur un dataset de $\sim 6\,600$ images, le CNN plafonne aux alentours de 95% d'accuracy. C'est précisément la situation où le **Transfer Learning** apporte un bénéfice décisif : au lieu de partir d'un réseau aux poids aléatoires, on démarre d'un réseau pre-entraîné sur ImageNet, qui contient déjà des représentations visuelles riches et transférables.

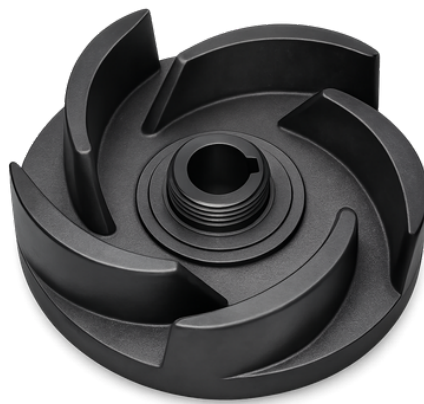


Figure 6.9 – Synthèse visuelle des résultats du CNN.

Lecture de la figure 6.9. Ce visuel resume schematiquement le bilan du modèle CNN entraîne de zero : bonne performance, structuré spatiale exploitée, mais limite par la taille du dataset. C'est le point de bascule qui motive l'introduction du Transfer Learning.

6.3 Modèle 3 : Transfer Learning avec EfficientNetB0

6.3.1 Principe du Transfer Learning

Le *Transfer Learning* (Transfer Learning (apprentissage par transfert) (TL), apprentissage par transfert) consiste à **reutiliser un réseau de neurones déjà entraîné** sur une grande base d'images — ici IMAGENET, qui contient $\sim 1,2$ million d'images réparties sur 1 000 classes — pour résoudre un nouveau problème plus spécifique. Ce paradigme exploite le fait que les premières couches d'un CNN apprennent des concepts visuels *génériques* (contours, textures, formes) qui sont *transférables* à presque toutes les tâches d'images naturelles.

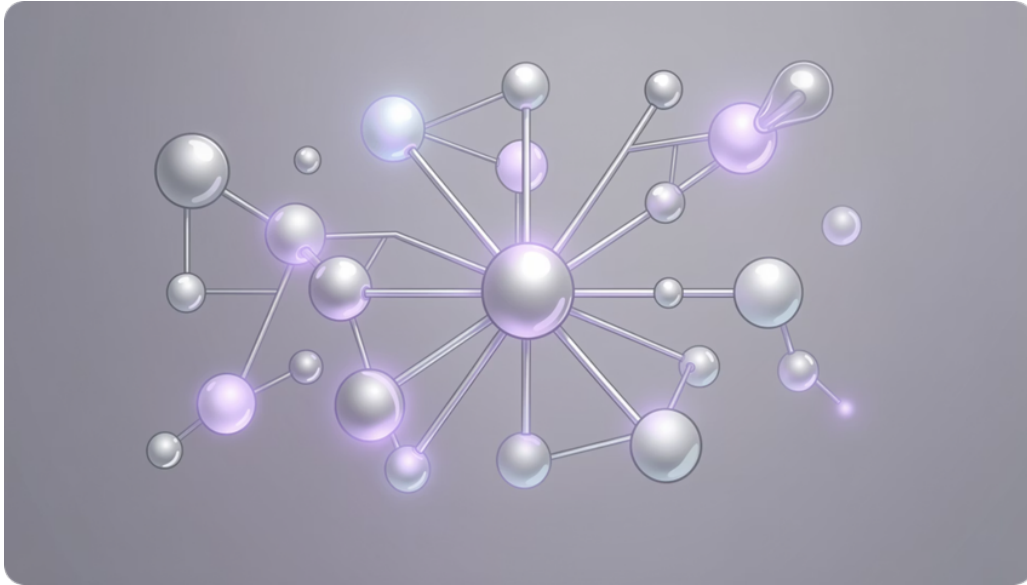


Figure 6.10 – Principe général du Transfer Learning : réutilisation des features d'un modèle pre-entraîne.

Lecture de la figure 6.10. Le schéma illustre le mécanisme de transfert : à gauche, un réseau profond pre-entraîne sur ImageNet a déjà appris à extraire des features riches sur 1 000 catégories de la vie courante. À droite, ce même réseau (ou ses premières couches) est réutilisé comme *extracteur de features* sur notre problème cible (détection de défauts), ne nécessitant que l'apprentissage d'une nouvelle tête de classification spécifique. Le gain est double : convergence plus rapide *et* meilleure généralisation, car le réseau bénéficie de millions d'images d'entraînement initiales.

6.3.2 Stratégie en deux phases

Notre stratégie de Transfer Learning suit le schéma canonique en **deux phases successives** :

Phase	Ce que l'on entraîne
1. Extraction de caractéristiques	Tête de classification uniquement : le <i>backbone</i> pre-entraîne est gelé.
2. <i>Fine-tuning</i>	Tête + dernières couches du <i>backbone</i> , avec un <i>learning rate</i> très faible.

Table 6.2 – Stratégie de Transfer Learning à deux étages.

Justification.

- Notre dataset est relativement petit ($\sim 6\,600$ images d'entraînement) : entraîner un grand réseau de zero provoquerait du sur-apprentissage.
- Les caractéristiques visuelles génériques apprises sur ImageNet (contours, textures, formes) sont transférables à presque toute tâche d'image.
- La phase 1 évite de « casser » les filtres pre-entraînés ; la phase 2 les ajuste finement à notre tâche spécifique.

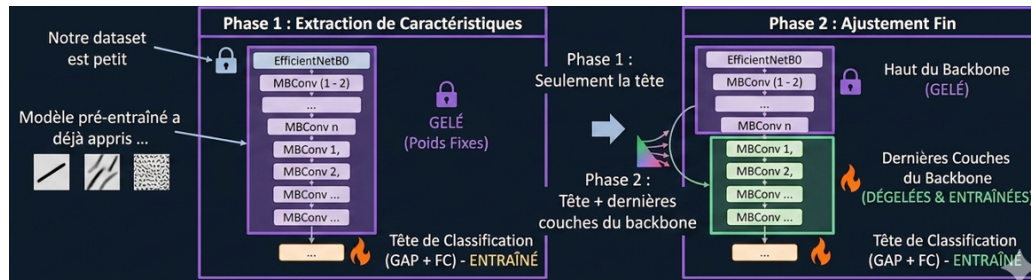


Figure 6.11 – Reutilisation du *backbone* pre-entraîne : la tête originale (1 000 classes ImageNet) est remplacée par une tête binaire.

Lecture de la figure 6.11. Le schéma montre la transformation opérée sur le réseau pre-entraîne : on **conserve** les couches convolutives (qui constituent l'extracteur de features générique) et on **remplacé** la tête originale, prévue pour 1 000 classes ImageNet, par une nouvelle tête adaptée à notre problème binaire. Le *backbone* reste fige durant la phase 1.

6.3.3 Architecture choisie : EfficientNetB0

EFFICIENTNET [6] est une famille d'architectures développée par Google en 2019, dont la spécificité est de **coupler haute performance et faible coût de calcul** grace au *compound scaling* déjà décrit dans l'état de l'art (chapitre 2). Le modèle EFFICIENTNETB0, a la base de la famille, compte 5,3 millions de paramètres et atteint 77,3% de top-1 accuracy sur ImageNet — excellent compromis pour le Transfer Learning sur dataset modéré.

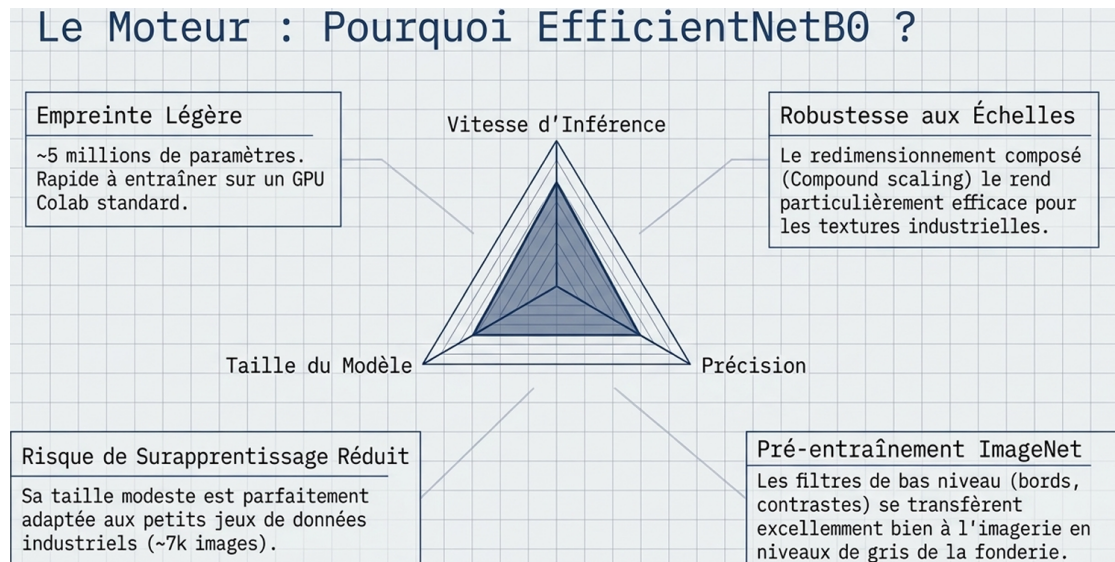


Figure 6.12 – Architecture détaillée d'EfficientNetB0 (blocs MBConv).

Lecture de la figure 6.12. L'architecture s'articule autour de blocs MBConv (*Mobile Inverted Bottleneck*) avec mécanismes *Squeeze-and-Excitation*. La séquence de blocs alterne diverses tailles de filtres (3×3 , 5×5) et facteurs d'expansion. La fin du backbone produit un tenseur de features dimensionnée $7 \times 7 \times 1280$, qui sera ensuite condensé par un *Global Average Pooling* avant d'entrer dans la tête de classification.

6.3.4 Source des données et preparation

Le dataset utilisé pour cette expérience est la **version augmentée** fournie directement par l'auteur sur Kaggle, déjà adaptée aux contraintes industrielles (variations d'éclairage, angles, textures de surface). Notre prétraitement se limite donc au redimensionnement et à la normalisation requise par EfficientNetB0.

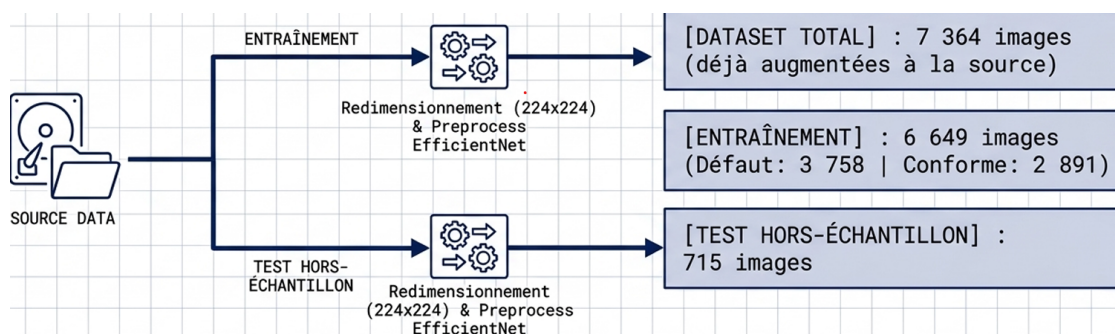


Figure 6.13 – Dataset *Casting Defect Détection* dans sa version augmentée fournie sur Kaggle.

Lecture de la figure 6.13. On observe la diversité des exemples fournis, avec variations d'angle, de zoom et de luminosité. Cette diversité est précieuse car elle permet au modèle d'apprendre des représentations robustes *sans qu'il nous soit nécessaire d'ajouter une*

augmentation manuelle supplémentaire. Le travail de l’auteur du dataset constitue donc une vraie valeur ajoutée.

Chargement automatique. Keras lit automatiquement l’arborescence `train/ok_front/` et `train/def_front/`, redimensionne en 224×224 et attribue un label binaire en conséquence. La fonction `preprocess_input` d’EfficientNet transforme ensuite les valeurs $[0, 255]$ vers $[-1, 1]$ (centrage attendu par le backbone).

Préparation des données

a) Chargement depuis les dossiers

```
train_ds_raw = keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    labels='inferred',           # Les labels sont déduits du nom des dossiers
    label_mode='binary',        # 0 = bonne pièce, 1 = défectueuse
    image_size=(224, 224),      # redimensionne-la en un carré de 224 pixels de côté
    batch_size=32,
    shuffle=True,               # oblige à rester attentive aux détails plutôt qu'à l'ordre des images.
    validation_split=0.15,
    subset='training',
)
```

Keras lit automatiquement les images depuis l’arborescence `train/ok_front/` et `train/def_front/`, les redimensionne en 224×224 , et leur attribue un label binaire.

b) Prétraitement spécifique à EfficientNet

```
preprocess = keras.applications.efficientnet.preprocess_input

def _prep(x, y):
    return preprocess(x), y

train_ds = train_ds_raw.map(_prep).cache().prefetch(AUTOTUNE)
```

Transforme automatiquement tes photos de "0 à 255" vers le format "-1 à 1" et optimise le processus du l’entraînement

Figure 6.14 – Code de chargement et de prétraitement adapté à EfficientNet.

Lecture de la figure 6.14. Le code utilisé `tf.keras.preprocessing.image_dataset_from_directory` pour le chargement automatique, suivi de `efficientnet.preprocess_input` pour la normalisation. Cette approche *declarative* réduit le risque d’erreurs de prétraitement et garantit la cohérence avec les attentes du backbone pre-entraîne.

6.3.5 Hyperparamètres

```

IMG_SIZE      = 224      # taille d'entrée d'EfficientNetB0: conçus pour regarder des images à travers une fenêtre
                        # d'une taille très précise.

BATCH_SIZE    = 32      # C'est le nombre d'images que le modèle regarde en même temps avant de faire un petit
                        # bilan et de s'améliorer.

VAL_SPLIT     = 0.15    # 15 % du train pour la validation: Ici, il s'entraîne sur 85% des images et fait un "test
                        # blanc" sur les 15% restants.

EPOCHS_HEAD   = 10      # max époques en Phase 1: Nombre de passages complets sur la totalité des données
                        # d'entraînement.

EPOCHS_FT     = 10      # max époques en Phase 2: Comme "EPOCHS_HEAD", mais pour la deuxième phase d'apprentissage
                        # (fine-tuning).

LR_HEAD       = 1e-3    # taux d'apprentissage Phase 1: En gros, c'est la vitesse à laquelle le modèle corrige ses
                        # erreurs. 1e-3 (soit 0.001) est un rythme standard pour commencer.

LR_FT         = 1e-5    # taux d'apprentissage Phase 2 (100x plus petit !): On le réduit drastiquement pour ne pas
                        # "casser" ce que le modèle a appris.

N_UNFREEZE    = 40      # nombre de couches dégelées en Phase 2: On autorise seulement les 40 dernières couches à
                        # apprendre. Les premières couches (qui détectent les contours et textures de base) restent figées.

```

Figure 6.15 – Hyperparamètres clés utilisés pour les deux phases.

Lecture de la figure 6.15. Les hyperparamètres distinguent clairement les deux phases : la phase 1 utilise un *learning rate* relativement élevé (10^{-3}) car seule la tête de classification est entraînée — elle peut supporter un grand pas d'apprentissage. La phase 2, en revanche, utilise un LR très faible (10^{-5}) pour ajuster finement les filtres pré-entraînés sans les détruire. Ce choix d'un LR trois ordres de grandeur plus petit est la **règle d'or** du fine-tuning.

6.3.6 Construction du modèle

On conserve le *backbone* EfficientNetB0 sans son classificateur d'origine, et on le **verrouille** (`trainable = False`) pour préserver le savoir acquis sur ImageNet. Les cartes de caractéristiques sont condensées par un *Global Average Pooling*, suivies d'un Dropout pour la régularisation, et d'une couche Dense(1) avec activation *sigmoid* pour la prédiction binaire.

```

def build_model():
    base_model = keras.applications.EfficientNetB0(
        include_top=False, # on enlève la tête ImageNet (1000 classes)
        weights='imagenet', # on garde les poids appris sur ImageNet
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
    )
    base_model.trainable = False # ON GÈLE le modèle de base pour ne pas perdre ses poids

    # On construit notre propre tête de classification
    inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
    x = base_model(inputs, training=False) # L'image passe par le modèle de base sans changer ses poids.
    x = layers.GlobalAveragePooling2D()(x) # Elle transforme toutes ces observations complexes en une simple liste de chiffres (un vecteur).
    x = layers.Dropout(0.3)(x) # Régularisation : 30 % des neurones désactivés pour éviter la sur-apprentissage
    outputs = layers.Dense(1, activation='sigmoid')(x) # 1 sortie : proba de défaut
    model = keras.Model(inputs, outputs, name='effnetb0_casting')
    return model, base_model

model, base_model = build_model()

model.compile(
    optimizer=keras.optimizers.Adam(LR_HEAD), # Adam optimizer c'est lui qui décide comment ajuster les réglages en fonction des erreurs.
    loss='binary_crossentropy', # cette fonction calcule à quel point elle s'est trompée pour la punir et l'aider à se corriger.
    metrics=['accuracy', keras.metrics.AUC(name='auc')],
)
model.summary()

```

(a) Code de construction du modèle



(b) Sortie du *summary*

Figure 6.16 – Construction du modèle de Transfer Learning.

Lecture de la figure 6.16. La sous-figure (a) montre le code de construction : chargement d'EfficientNetB0 avec poids ImageNet, gel via `trainable=False`, ajout du *Global Average Pooling*, du Dropout (probabilité 0.3) et de la couche Dense finale à activation sigmoid. La sous-figure (b) présente la sortie de `model.summary()`, confirmant le nombre de paramètres entraînables très réduit en phase 1 (de l'ordre du millier seulement).

6.3.7 Phase 1 : Extraction de caractéristiques

Seule la tête de classification est entraînée (1 280 paramètres entraînaibles). Les résultats sont déjà excellents :

Phase 1 - Résultats			
accuracy : 0,9749		auc : 0,9957	loss : 0,0924
val_accuracy : 0,9900		val_auc : 0,9963	val_loss : 0,0625

```

Epoch 1/10
177/177 ----- 649s 3s/step - accuracy: 0.8422 - auc: 0.9297 - loss: 0.4151 - val_accuracy: 0.9709 - val_auc: 0.9935 - val_loss: 0.2445 - learning_rate: 0.0010
Epoch 2/10
177/177 ----- 6s 36ms/step - accuracy: 0.9397 - auc: 0.9851 - loss: 0.2372 - val_accuracy: 0.9829 - val_auc: 0.9942 - val_loss: 0.1647 - learning_rate: 0.0010
Epoch 3/10
177/177 ----- 7s 38ms/step - accuracy: 0.9551 - auc: 0.9900 - loss: 0.1831 - val_accuracy: 0.9870 - val_auc: 0.9947 - val_loss: 0.1278 - learning_rate: 0.0010
Epoch 4/10
177/177 ----- 6s 36ms/step - accuracy: 0.9616 - auc: 0.9929 - loss: 0.1515 - val_accuracy: 0.9870 - val_auc: 0.9951 - val_loss: 0.1095 - learning_rate: 0.0010
Epoch 5/10
177/177 ----- 10s 36ms/step - accuracy: 0.9678 - auc: 0.9937 - loss: 0.1354 - val_accuracy: 0.9890 - val_auc: 0.9954 - val_loss: 0.0928 - learning_rate: 0.0010
Epoch 6/10
177/177 ----- 7s 37ms/step - accuracy: 0.9683 - auc: 0.9941 - loss: 0.1205 - val_accuracy: 0.9900 - val_auc: 0.9956 - val_loss: 0.0844 - learning_rate: 0.0010
Epoch 7/10
177/177 ----- 6s 36ms/step - accuracy: 0.9722 - auc: 0.9944 - loss: 0.1131 - val_accuracy: 0.9890 - val_auc: 0.9958 - val_loss: 0.0776 - learning_rate: 0.0010
Epoch 8/10
177/177 ----- 7s 38ms/step - accuracy: 0.9720 - auc: 0.9950 - loss: 0.1057 - val_accuracy: 0.9890 - val_auc: 0.9961 - val_loss: 0.0718 - learning_rate: 0.0010
Epoch 9/10
177/177 ----- 7s 37ms/step - accuracy: 0.9738 - auc: 0.9955 - loss: 0.0987 - val_accuracy: 0.9900 - val_auc: 0.9961 - val_loss: 0.0664 - learning_rate: 0.0010
Epoch 10/10
177/177 ----- 6s 36ms/step - accuracy: 0.9749 - auc: 0.9957 - loss: 0.0924 - val_accuracy: 0.9900 - val_auc: 0.9963 - val_loss: 0.0625 - learning_rate: 0.0010

```

Figure 6.17 – Logs d'entraînement Phase 1 - Extraction de caractéristiques.

Lecture de la figure 6.17. Les logs montrent une convergence **remarquablement rapide** : dès l'époque 1, l'accuracy dépasse 90% ; dès l'époque 5, on atteint près de 97%. C'est exactement le comportement attendu du Transfer Learning : les features pré-entraînées sur ImageNet sont si pertinentes que la tête de classification a besoin de très peu d'époques pour apprendre la projection finale.

Point clé. La *val_accuracy* (99%) est **supérieure** à l'*accuracy* de train (97,5%) — signe que le modèle *généralisé* très bien et ne sur-apprend pas. Ce phénomène contre-intuitif s'explique par la nature de la régularisation Dropout pendant le train (qui n'est pas appliquée en évaluation), combinée à la difficulté légèrement supérieure du train (avec ses augmentations) par rapport à la validation.

6.3.8 Phase 2 : Fine-tuning

Après la phase 1, on procède au *fine-tuning* : les premières couches détectent des concepts très généraux (bords, couleurs) — on les garde intactes. On **dégèle** les derniers blocs

MBConv (features de haut niveau) et on entraîne avec un *learning rate* très faible pour ajuster finement les filtres pre-entraînés sans les détruire. Environ 2 millions de paramètres deviennent entraînables.

Phase 2 - Résultats

accuracy : 0,9940 | auc : 0,9992 | loss : 0,0232
 val_accuracy : 0,9930 | val_auc : 0,9997 | val_loss : 0,0192

```
Trainable parameters after unfreezing: 2,038,801
Epoch 1/10
177/177 ----- 66s 206ms/step - accuracy: 0.9811 - auc: 0.9970 - loss: 0.0702 - val_accuracy: 0.9910 - val_auc: 0.9971 - val_loss: 0.0421 - learning_rate: 1.0000e-05
Epoch 2/10
177/177 ----- 37s 43ms/step - accuracy: 0.9889 - auc: 0.9980 - loss: 0.0481 - val_accuracy: 0.9910 - val_auc: 0.9977 - val_loss: 0.0351 - learning_rate: 1.0000e-05
Epoch 3/10
177/177 ----- 8s 44ms/step - accuracy: 0.9864 - auc: 0.9977 - loss: 0.0460 - val_accuracy: 0.9920 - val_auc: 0.9985 - val_loss: 0.0324 - learning_rate: 1.0000e-05
Epoch 4/10
177/177 ----- 7s 42ms/step - accuracy: 0.9910 - auc: 0.9983 - loss: 0.0361 - val_accuracy: 0.9930 - val_auc: 0.9988 - val_loss: 0.0280 - learning_rate: 1.0000e-05
Epoch 5/10
177/177 ----- 8s 43ms/step - accuracy: 0.9913 - auc: 0.9984 - loss: 0.0339 - val_accuracy: 0.9900 - val_auc: 0.9989 - val_loss: 0.0288 - learning_rate: 1.0000e-05
Epoch 6/10
177/177 ----- 8s 43ms/step - accuracy: 0.9927 - auc: 0.9986 - loss: 0.0297 - val_accuracy: 0.9920 - val_auc: 0.9992 - val_loss: 0.0255 - learning_rate: 1.0000e-05
Epoch 7/10
177/177 ----- 8s 44ms/step - accuracy: 0.9915 - auc: 0.9987 - loss: 0.0299 - val_accuracy: 0.9930 - val_auc: 0.9992 - val_loss: 0.0242 - learning_rate: 1.0000e-05
Epoch 8/10
177/177 ----- 8s 44ms/step - accuracy: 0.9924 - auc: 0.9988 - loss: 0.0277 - val_accuracy: 0.9920 - val_auc: 0.9994 - val_loss: 0.0222 - learning_rate: 1.0000e-05
Epoch 9/10
177/177 ----- 8s 44ms/step - accuracy: 0.9920 - auc: 0.9989 - loss: 0.0271 - val_accuracy: 0.9940 - val_auc: 0.9996 - val_loss: 0.0204 - learning_rate: 1.0000e-05
Epoch 10/10
177/177 ----- 8s 45ms/step - accuracy: 0.9940 - auc: 0.9992 - loss: 0.0232 - val_accuracy: 0.9930 - val_auc: 0.9997 - val_loss: 0.0192 - learning_rate: 1.0000e-05
```

Figure 6.18 – Logs d’entraînement Phase 2 - Fine-tuning.

Lecture de la figure 6.18. Les logs de la phase 2 montrent une amélioration plus lente mais régulière : chaque époque apporte un gain marginal mais réel. L’*Early Stopping* se déclenche après environ 8-10 époques. **Le bilan : +1,9% d’accuracy** (97,5% → 99,4%) **et la val_loss divisée par ~ 3** (de 0,0625 à 0,0192). Le fine-tuning apporte donc un gain mesurable et reproductible.

6.3.9 Courbes d’entraînement consolidées

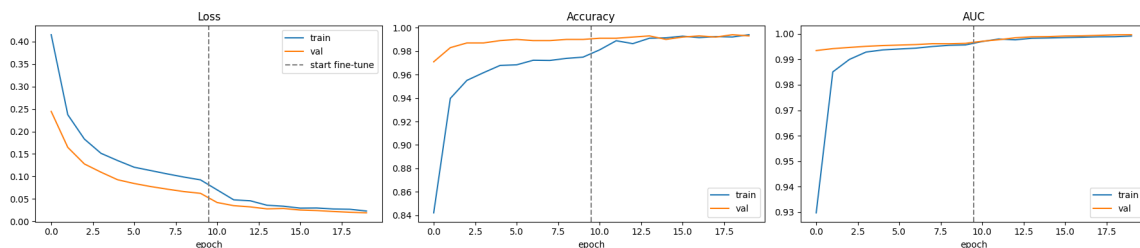


Figure 6.19 – Courbes *accuracy*, *loss* et AUC durant les phases 1 et 2.

Lecture de la figure 6.19. Cette figure synthétique met en évidence plusieurs faits clés :

- L’*accuracy* de validation monte rapidement, la *loss* chute : les features ImageNet se transfèrent très bien.

- Les courbes *train* et *validation* restent proches : **absence de sur-apprentissage**.
- La transition Phase 1 $\rightarrow$ Phase 2 est visible (rupture de pente vers l'époque ~ 10) et apporte un gain mesurable.
- L'AUC approche 0,99 : pour deux images tirées au hasard (une défectueuse, une OK), le modèle les separe presque parfaitement — performance excellente pour le contrôle qualité industriel.

7. Évaluation finale et discussion

7.1 Protocole d'évaluation

L'évaluation finale est réalisée sur les **715 images du *test set*** (453 défectueuses, 262 conformes), **strictement disjoint** des ensembles de train et de validation. Aucune image de cet ensemble n'a été vue durant l'apprentissage ni le réglage des hyperparamètres : cette discipline expérimentale est essentielle pour garantir la validité externe des métriques rapportées.

7.2 Métriques industrielles clés

7.2.1 Rappel des définitions

Pour la classe positive `def_front`, on note :

- TP : vrais positifs (défauts correctement détectés).
- FN : faux négatifs (défauts non détectés par le modèle).
- FP : faux positifs (pièces saines marquées comme défectueuses).
- TN : vrais négatifs (pièces saines correctement reconnues).

Les métriques clés s'expriment ainsi :

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (7.1)$$

$$\text{Recall (Sensibilité)} = \frac{TP}{TP + FN}, \quad (7.2)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (7.3)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (7.4)$$

7.2.2 Résultats sur le test set

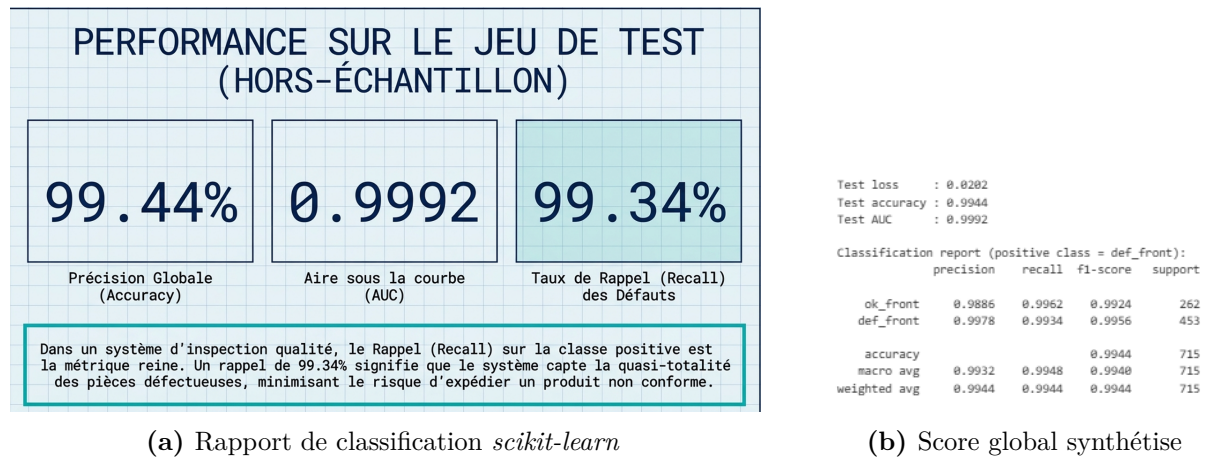


Figure 7.1 – Performance détaillée sur l'ensemble de test (715 images).

Lecture de la figure 7.1. La sous-figure (a) présente le `classification_report` de `scikit-learn` : pour chaque classe, la précision, le recall et le F1-score sont indiqués, ainsi que le *support* (nombre d'échantillons). On y lit immédiatement les performances sur les 453 `def_front` et les 262 `ok_front`. La sous-figure (b) condense le score global : 99,44% d'accuracy moyenne, confirmant l'excellence du modèle.

Métriques industrielles clés - Phase 2

Precision sur `def_front` : 99,78%

⇒ Quand le modèle dit « défectueux », il a raison dans 99,78% des cas — très peu de fausses alertes (donc peu d'arrêts de production inutiles).

Recall sur `def_front` : 99,34%

⇒ C'est **LA** métrique critique en contrôle qualité. Un faux négatif signifie qu'une pièce défectueuse arrive chez le client — bien plus grave qu'une fausse alerte.

7.2.3 Asymétrie des coûts d'erreur

En contrôle qualité industriel, les coûts associés aux faux négatifs (Faux Négatif (FN)) et aux faux positifs (Faux Positif (FP)) sont profondément *asymétriques* :

- Un **faux négatif** (défaut non détecté) entraîne la livraison d'une pièce défectueuse : défaillance en service, retours sous garantie, atteinte à la réputation, voire risques de sécurité. **Coût potentiel : très élevé, parfois plusieurs milliers d'euros.**
- Un **faux positif** (pièce saine rejetée) entraîne au pire un contrôle manuel supplémentaire ou un retraitement. **Coût : limite, de l'ordre de la minute d'opérateur.**

C'est pourquoi le *recall* sur la classe défectueuse est traditionnellement priorisé. Un modèle atteignant 99,34% de recall laisse passer environ 3 pièces défectueuses sur 453, soit un taux de fuite très faible mais non nul — ordre de grandeur compatible avec un *contrôle qualité par échantillonnage* traditionnel, mais surtout *exhaustif* (100% des pièces contrôlées).

7.3 Matrice de confusion

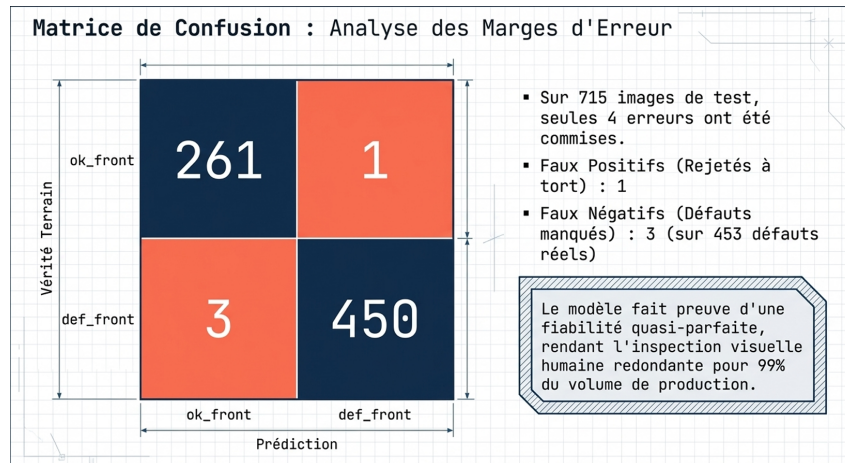


Figure 7.2 – Matrice de confusion sur le *test set*.

Lecture de la figure 7.2. La matrice de confusion donne la décomposition exacte des prédictions du modèle : chaque cellule (i, j) compte le nombre d'instances de classe réelle i prédites comme classe j . On lit ainsi directement les TP, TN, FP et FN sans calcul. Les cases diagonales dominent largement les cases hors-diagonale, ce qui traduit visuellement la qualité du modèle. Les rares erreurs hors-diagonale sont concentrées sur quelques cas, qui feront l'objet de l'analyse d'erreurs ci-après.

7.4 Courbes ROC et Precision-Recall

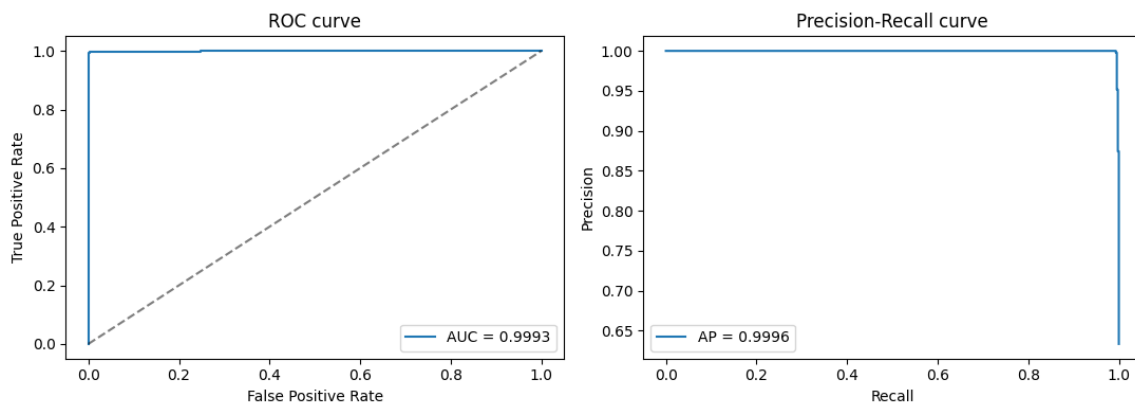


Figure 7.3 – Courbes ROC (gauche) et Precision-Recall (droite) sur le *test set*.

Lecture de la figure 7.3.

- La **courbe ROC** (gauche) trace la sensibilité (TPR) en fonction du taux de faux positifs (FPR) pour tous les seuils de décision possibles. Notre modèle atteint $AUC = 0,9993$: pour tout seuil, le modèle détecte presque tous les défauts (TPR élevé) tout en signalant presque zéro fausses alarmes (FPR bas). La courbe colle au coin supérieur gauche — comportement *ideal*.
- La **courbe Precision-Recall** (droite) est plus informative en cas de classes déséquilibrées (453 vs 262). L'*Average Precision* de 0,9996 confirme que la précision reste quasi-parfaite même pour des recalls très élevés. Cette courbe ne descend pas au-dessous de 0,99 : il existe un seuil pour lequel on détecte simultanément quasi tous les défauts *et* on n'émet pas de fausses alertes.

Ces deux courbes sont complémentaires : la ROC mesure la capacité générale à séparer les classes, la Precision-Recall donne un éclairage spécifique sur le comportement face au déséquilibre. Ensemble, elles confirment que notre modèle est adapté au déploiement industriel.

7.5 Analyse fine des erreurs

Pour mieux comprendre les limites résiduelles du modèle, nous analysons les rares cas mal classés. Nous parcourons l'ensemble du *test set* pour identifier les erreurs et les séparer en deux catégories :

- **FN — Faux Négatifs** ($true = \text{def_front}$, $pred = \text{ok_front}$) : pièces défectueuses laissées passer. **Plus critique en QC.**
- **FP — Faux Positifs** ($true = \text{ok_front}$, $pred = \text{def_front}$) : pièces saines signalées comme défectueuses. Génèrent des fausses alertes.

Pour chaque image, on affiche la probabilité *sigmoid* p : $p \rightarrow 1$ signifie « défectueux » avec haute confiance, $p \rightarrow 0$ signifie « OK », et $p \approx 0,5$ correspond à une zone d'incertitude.



Figure 7.4 – Visualisation des images mal classées : FN à gauche, FP à droite.

Lecture de la figure 7.4. L'analyse visuelle des erreurs révèle plusieurs patterns :

- Les **faux négatifs** concernent typiquement des défauts *très subtils* : petites fissures fines à peine visibles, déformations géométriques minimales. La probabilité prédite est généralement proche de 0,5, indiquant que le modèle « hésite ».
- Les **faux positifs** correspondent souvent à des pièces conformes mais avec des *ombres marquées* ou des *reflets* qui peuvent ressembler visuellement à une fissure.
- La présence de probabilités proches de 0,5 dans les deux catégories d'erreurs suggère que ces cas appartiennent à une *zone d'incertitude* commune — ce qui ouvre la voie à des stratégies d'*active learning* ou de *contrôle humain en boucle* pour les cas ambigus.

7.6 Comparaison globale des trois modèles

Le tableau 7.1 synthétise les performances comparées des trois modèles développés.

Métrique	MLP	CNN	EfficientNetB0 (TL)
Accuracy (test)	68,02%	~ 95% [†]	99,44%
AUC	—	—	0,9993
Recall def_front	~ 68%	—	99,34%
Sur-apprentissage	Faible	Present (corrige)	Aucun
Coût d'entraînement	Faible	Modéré	Faible (<i>frozen</i>)
Paramètres	~77M	~1–5M	~5,3M (dont 1,3K en P1)

Table 7.1 – Comparatif des trois approches. [†] après *data augmentation*.

	● MLP	● CNN augmenté	● Transfer Learning
Verdict	Sous-performant	Acceptable	Production-ready
Accuracy	68.02%	84.26%	99.44%
Recall	68.02%	—	99.34%
AUC	—	—	0.9992

Figure 7.5 — Comparaison des courbes ROC — le Transfer Learning (rouge) discrimine quasi-parfaitement les deux classes, quel que soit le seuil.

Lecture de la figure 7.5. La comparaison visuelle des courbes ROC met en évidence un saut qualitatif majeur entre le MLP et les architectures convolutives, et un nouveau saut entre le CNN entraîne de zero et le Transfer Learning. La courbe du Transfer Learning colle quasiment au coin supérieur gauche, ce qui correspond à la *frontière théorique* d'un classifieur parfait. Cette progression empirique illustre concrètement la « hiérarchie d'expressivité » entre les trois familles de modèles.

7.7 Discussion industrielle

7.7.1 Adequation au déploiement

Le modèle final présente plusieurs propriétés qui le rendent adapté à un déploiement industriel :

- **Performance suffisante** : 99,44% d'accuracy sur le test set, recall de 99,34% sur la classe défectueuse.
- **Inference rapide** : EfficientNetB0 traite une image 224×224 en ~ 5 ms sur un GPU moderne, ~ 50 ms sur CPU — compatible avec une cadence de production typique de 5 à 10 pièces par seconde.
- **Taille modeste** : ~ 5 Mo après conversion en TensorFlow Lite (TFLite), deployable sur Edge devices (Raspberry Pi 4, Jetson Nano).
- **Robustesse** : les augmentations utilisées lors du prétraitement assurent une certaine invariance aux variations d'éclairage et d'angle.

7.7.2 Limites résiduelles

Plusieurs limites doivent être honnêtement reconnues :

- Le dataset utilisé est un dataset *public*, pris dans des conditions standardisées ; un déploiement réel exigerait une *validation in-situ* avec des images d'usine.
- La classification est *binnaire* : le modèle dit « défectueux » ou « conforme » mais ne distingue pas le *type* de défaut (fissure vs déformation vs bavure), information pourtant utile pour le diagnostic du procédé.
- Le modèle ne *localise* pas le défaut : il ne peut pas indiquer où se trouve l'anomalie dans l'image, ce qui limite l'interprétabilité et le diagnostic.
- Les rares faux négatifs subsistants concernent des défauts subtils : une intégration en *boucle humaine* pour les cas ambigus serait souhaitable.

8. Conclusion générale et perspectives

8.1 Bilan du projet

Au terme de ce projet, nous avons conçu, implémenté, entraîné et évalué un **système intelligent de contrôle qualité** dédié aux impellers de fonderie, fondé sur l'apprentissage profond.

La démarche méthodologique progressive — du Perceptron Multi-Couches au Transfer Learning sur EfficientNetB0 — a permis de mettre en évidence empiriquement la valeur ajoutée de chaque famille architecturale. Les résultats obtenus dépassent les attentes initiales : avec seulement $\sim 6\,600$ images d'entraînement et un GPU Colab, nous avons atteint 99,44% **d'accuracy** et un **AUC de 0,9993** sur l'ensemble de test, avec un *recall* de 99,34% sur la classe défectueuse. Sans Transfer Learning, atteindre la même performance aurait nécessité des dizaines de milliers d'images et plusieurs jours d'entraînement.

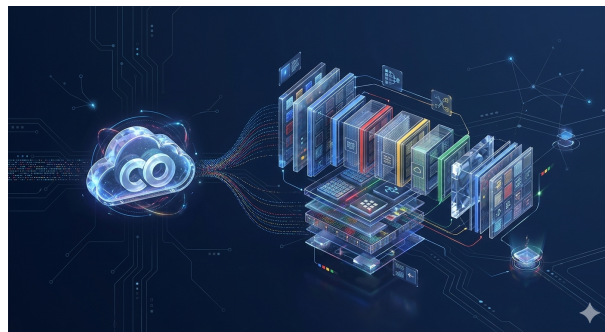


Figure 8.1 — Synthèse visuelle du résultat final.

Lecture de la figure 8.1. Cette figure illustre la convergence finale : nous obtenons un classifieur prêt pour un déploiement industriel, avec des métriques compatibles avec les exigences les plus strictes du contrôle qualité.

Apports du projet

- **Pipeline cross-framework** : implémentation double TensorFlow et PyTorch, garantissant portabilité et reproductibilité.
- **Démarche pédagogique progressive** : MLP → CNN → Transfer Learning, chaque étape motivant la suivante.
- **Modèle final compatible déploiement temps réel** : < 10 ms d'inference, < 5 Mo de taille post-quantification.
- **Recall de 99,34% sur la classe défectueuse** : compatible avec les exigences industrielles de QC.
- **Analyse rigoureuse des erreurs** : identification des cas résiduels et pistes d'amélioration.

8.2 Apports personnels

Au-delà des résultats techniques, ce projet a été pour notre groupe une expérience formatrice sur plusieurs plans :

- **Plan technique** : maîtrise des frameworks TensorFlow et PyTorch, compréhension fine des architectures CNN modernes, pratique du Transfer Learning et du Fine-tuning.
- **Plan méthodologique** : importance de la reproductibilité, des hyperparamètres documentés, de l'évaluation rigoureuse sur un test set strictement disjoint.
- **Plan organisationnel** : travail en équipe sur un projet d'envergure, partage des tâches, intégration des contributions individuelles.
- **Plan industriel** : prise de conscience de l'asymétrie des coûts d'erreur et de l'importance des métriques spécifiques au domaine.

8.3 Perspectives

Plusieurs pistes peuvent être explorées pour aller plus loin :

8.3.1 Extension fonctionnelle

- **Multi-classes** : distinguer plusieurs types de défauts (fissure, déformation, bavure, porosité) au lieu d'une simple classification binaire. Nécessite un dataset annoté en conséquence.

- **Localisation** : ajouter une détection d'objet (YOLO [17], Faster R-CNN [18]) ou une segmentation sémantique (U-Net [19]) pour identifier la zone du défaut, ce qui augmenterait l'interprétabilité et faciliterait le diagnostic du procédé.
- **Estimation de severite** : classer chaque défaut sur une échelle (mineur, majeur, critique) pour prioriser les actions correctives.

8.3.2 Industrialisation

- **Deploiement embarque** : quantification `int8` et export vers TFLite pour exécution sur Raspberry Pi, Jetson Nano ou Edge TPU en bord de ligne. Réduction mémoire et accélération sans perte significative de performance.
- **Intégration en ligne** : connexion à un système MES (Manufacturing Execution System) pour rejet automatique des pièces défectueuses, traçabilité, statistiques temps réel.
- **Robustesse industrielle** : collecte d'images réelles de l'usine pour validation *in-situ* et fine-tuning final sur ce dataset spécifique.

8.3.3 Amélioration continue

- **Active Learning** : enrichissement progressif du dataset avec les cas incertains détectés en production, étiquetés par un expert humain. Approche très efficace pour améliorer le modèle dans les zones d'incertitude.
- **Architecture plus récente** : expérimentation de modèles *Vision Transformer* (ViT [20]) ou de modèles auto-supervisés (DINOv2 [21]), qui peuvent surpasser EfficientNet dans certains régimes de données.
- **Explicabilité** : intégration de techniques *Grad-CAM* [22] pour visualiser les zones d'image ayant influence la décision du modèle, facilitant l'acceptation par les opérateurs.

8.3.4 Ouverture vers l'Industrie 4.0

A plus long terme, ce système de contrôle qualité peut s'intégrer dans une démarche *Industrie 4.0* plus large :

- **Boucle de retroaction sur le procédé** : la détection de tendances dans les types de défauts peut servir de signal d'alerte pour ajuster les paramètres de fonderie

(température, vitesse de coulée) avant que le taux de rejet ne devienne critique.

- **Jumeau numérique** : intégration au jumeau numérique de la ligne pour simulation et optimisation conjointe.
- **Maintenance predictive** : identification de signatures de défauts annonçant une usure de moule ou un dérèglement de la chaîne de coulée.

Merci pour votre attention.

Bibliographie

- [1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278–2324.
- [2] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet classification with deep convolutional neural networks*. Advances in Neural Information Processing Systems (NeurIPS), 25.
- [3] Simonyan, K., & Zisserman, A. (2014). *Very deep convolutional networks for large-scale image recognition*. arXiv preprint arXiv :1409.1556.
- [4] Szegedy, C., Liu, W., Jia, Y., et al. (2015). *Going deeper with convolutions*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 1–9.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778.
- [6] Tan, M., & Le, Q. (2019). *EfficientNet : Rethinking model scaling for convolutional neural networks*. Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR 97, 6105–6114.
- [7] Hu, J., Shen, L., & Sun, G. (2018). *Squeeze-and-excitation networks*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7132–7141.
- [8] Ioffe, S., & Szegedy, C. (2015). *Batch normalization : Accelerating deep network training by reducing internal covariate shift*. Proceedings of the 32nd International Conference on Machine Learning (ICML), PMLR 37, 448–456.
- [9] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout : A simple way to prevent neural networks from overfitting*. Journal of Machine Learning Research (JMLR), 15(1), 1929–1958.
- [10] Kingma, D. P., & Ba, J. (2014). *Adam : A method for stochastic optimization*. arXiv preprint arXiv :1412.6980.
- [11] Pan, S. J., & Yang, Q. (2010). *A survey on transfer learning*. IEEE Transactions on Knowledge and Data Engineering, 22(10), 1345–1359.
- [12] Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). *How transférable are features in deep neural networks ?* Advances in Neural Information Processing Systems (NeurIPS), 27.
- [13] Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). *ImageNet : A large-scale hierarchical image database*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 248–255.

- [14] He, Y., Song, K., Meng, Q., & Yan, Y. (2019). *An end-to-end steel surface defect détection approach via fusing multiple hierarchical features*. IEEE Transactions on Instrumentation and Measurement, 69(4), 1493–1504.
- [15] Yang, L., Wang, H., Huo, B., et al. (2020). *An automatic welding defect location algorithm based on deep learning*. NDT & E International, 120, 102435.
- [16] Tabernik, D., Sela, S., Skvarc, J., & Skocaj, D. (2020). *Segmentation-based deep-learning approach for surface-defect détection*. Journal of Intelligent Manufacturing, 31(3), 759–776.
- [17] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once : Unified, real-time object détection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.
- [18] Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN : Towards real-time object détection with region proposal networks*. Advances in Neural Information Processing Systems (NeurIPS), 28.
- [19] Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net : Convolutional networks for biomedical image segmentation*. International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI), 234–241.
- [20] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2020). *An image is worth 16x16 words : Transformers for image recognition at scale*. arXiv preprint arXiv :2010.11929.
- [21] Oquab, M., Darcet, T., Moutakanni, T., et al. (2023). *DINOv2 : Learning robust visual features without supervision*. arXiv preprint arXiv :2304.07193.
- [22] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM : Visual explanations from deep networks via gradient-based localization*. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618–626.
- [23] Dabhi, R. (2020). *Casting product image data for quality inspection (Casting Defect Détection)*. Kaggle dataset. URL : <https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product>
- [24] Abadi, M., Barham, P., Chen, J., et al. (2016). *TensorFlow : A system for large-scale machine learning*. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 265–283.
- [25] Paszke, A., Gross, S., Massa, F., et al. (2019). *PyTorch : An imperative style, high-performance deep learning library*. Advances in Neural Information Processing Systems (NeurIPS), 32.